

从 C++ 到计算系统：第一册实践卷

可执行文件、ABI、Linux 工具链与可信测量

CPU Performance Study

本卷是《从 C++ 到计算系统：第一册》的配套实践卷，专门承接习题、研究项目、完整代码、测试、benchmark 和机器证据报告。

前言

第一册正文讲清了 C++ 程序怎样经过编译、链接、装载、ABI 约定、指令执行和 Linux 工具链观察，最终成为一个正在运行的进程。但只读正文容易产生一种错觉：好像看懂概念就等于会做工程。真正的门槛在证据链里：你能不能拿一个自己编译出来的二进制，说明它是什么格式，入口在哪里，符号落在哪些 section，函数调用遵守什么 ABI，benchmark 结果是否被 checksum 保护，性能结论能否被反汇编、工具输出或测试复查。

本卷的贯穿项目叫 **Executable Evidence Kit**（可执行文件证据工具箱）。它不是一个玩具读文件程序，而是一条逐步扩展的研究项目：第一阶段读取字节并建立 checksum；第二阶段解析 ELF64 header；第三阶段解析 section table 和 string table；第四阶段解析 symbol table；第五阶段输出 report 和 CSV；第六阶段把 CLI、GTest/GMock、Google Benchmark、readelf/objdump/perf 观察和研究报告串起来。每一阶段都要回答同一个问题：这条结论来自哪里，怎样复查，当前工具还不能证明什么。

本卷补的是第一册最缺的两类训练：一类是章节习题，每章都要落到具体代码、命令和证据；另一类是研究项目，读者要从“照着跑”走到“能解释一个真实二进制”。因此本卷不会只列题目，也不会只贴模板。每个任务都先给问题背景，再说明最小实现、失败边界、测试要求和报告要求。

核心理想

第一册实践卷的目标不是把 ELF、ABI、汇编和 benchmark 各讲一次，而是让它们在同一个工程里互相校验：字节是事实，解析是解释，测试固定合同，工具输出交叉验证，报告只写证据能支撑的结论。

常见缩写和术语先导

本卷从第一章开始就会同时出现 C++、ELF、ABI、section、symbol、PMU、benchmark、GTest 等词。不要把它们当成孤立名词背下来；每个词都对应代码中的一个对象、命令行输出中的一个字段，或报告中的一条证据。

CPU: Central Processing Unit 中央处理器。它执行机器指令、访问寄存器和内存，并通过缓存、流水线、分支预测和乱序执行影响程序速度。

ELF: Executable and Linkable Format Linux 常见的可执行文件和目标文件格式。本卷只实现 ELF64 小端切片，重点训练 header、section、string table 和 symbol table 的证据链。

ABI: Application Binary Interface 应用二进制接口。它约定函数参数、返回值、寄存器保存、栈对齐、符号和链接边界，让不同编译单元能在二进制层面协作。

ISA: Instruction Set Architecture 指令集架构。x86-64、AArch64、RISC-V 都是 ISA；ELF header 的 machine 字段能告诉我们二进制面向哪类机器。

x86-64 64 位 x86 指令集。本卷第一版以 x86-64 为主要观察对象，但解析器会把 AArch64 和 RISC-V machine 字段也分类出来。

section ELF 文件中的一段逻辑区域，例如 `.text`、`.rodata`、`.symtab` 和 `.strtab`。section 是链接器和调试工具常用的组织方式。

symbol 符号。函数名、全局对象或链接器可见标记都可能成为符号。符号把源码名字、section、地址、大小和绑定关系连接起来。

string table 字符串表。ELF 中很多名字不直接写在结构体里，而是保存一个偏移，偏移指向某个字符串表。

entry address 入口地址。程序开始执行的虚拟地址，不等于 C++ 的 `main` 函数地址；运行时启动代码会先执行。

CLI: Command-Line Interface 命令行接口。本卷的 `cpu1ee_cli` 用文件路径和模式参数输出 report 或 section CSV。

CSV: Comma-Separated Values 逗号分隔表格。适合保存 section 摘要、benchmark 结果和可脚本复查的字段。

checksum 校验摘要。本卷使用它固定输入字节身份，避免报告只描述文件名而没有描述内容。

reference 参考实现。可以慢，但必须确定、清楚、可复查。实践中先有 reference，再让优化或工具扩展与它对齐。

oracle 判定器。它比较 reference 和待测实现，判断输出是否满足同一语义。

contract 合同。代码边界的明确约定：输入是什么，输出是什么，错误怎样表达，哪些行为暂不支持。

GTest: GoogleTest C++ 单元测试框架。本卷用它固定 ELF fixture、错误输入、符号上限和格式化输出。

GMock: GoogleMock GoogleTest 配套 mock 框架。复杂工程可用它替换外部依赖；本卷先通过 GTest/GMock 工程接入建立测试能力。

Google Benchmark C++ benchmark 库。本卷用它做解析路径 smoke，不用 benchmark 替代正确性测试。

PMU: Performance Monitoring Unit 性能监控单元。它统计 retired instructions、cache miss、branch miss 等硬件事件，是性能报告的重要证据来源。

perf Linux 性能分析工具，可读取 PMU 事件、采样热点和统计执行行为。没有

`perf` 权限时，报告要写明限制。

objdump 二进制工具，可反汇编、查看符号和 `section` 信息。它常用来把 C++ 函数和机器指令对应起来。

readelf ELF 检查工具，可查看 ELF header、section table、symbol table 等字段。它是本卷解析器的交叉验证对象。

Debug/Release 常见构建类型。Debug 偏调试信息和可读性；Release 打开优化。两者的机器码和性能结论不能混用。

本卷结构

本卷按第一册正文逐章对应组织，不再按项目阶段另起目录。读者读完第一册某一章，可以直接翻到实践卷同编号章节：第 0 章实践对应原书第 0 章心智模型，第 1 章实践对应程序到进程，第 5 章实践对应编译器 IR 与优化证据，后续 x86-64、ABI、测量章节也保持原书顺序。这样目录本身就是学习路线，不需要读者在“原书章节”和“项目阶段”之间来回映射。

贯穿项目仍然是 **Executable Evidence Kit**（可执行文件证据工具箱），代码目录仍然是：

```
1 books/cpu-volume-1-practice/labs/executable_evidence
```

区别在于项目推进方式改成“原书章节驱动”：心智模型章先建立证据链，程序到进程章再建立构建产物身份，编译器章比较 Debug/Release，CPU 执行章把 `.text` 和反汇编连起来，内存章解释 `offset`、`size` 和小端读取，Linux 工具章把 CLI 与 `readelf/objdump` 对齐，x86 与 ABI 章节再把指令、符号、调用约定落到同一个二进制上，最后用测量章节收束成可信 benchmark 和研究报告。

对应关系如下。第 0 章实践是学习地图实践，交付一张源码、工具、ELF、进程、测量的证据链。第 1 章实践是程序到进程实践，交付 `byte size`、`checksum`、`machine`、`entry` 的产物身份记录。第 5 章实践是 IR 与优化实践，交付 Debug/Release 二进制证据比较。第 2 章实践把函数符号、`.text` 和反汇编对应起来。第 3 章实践验证小端读取、范围检查和 `section` 边界。第 4 章实践用 `readelf`、`objdump`、`ctest` 交叉验证。

第二部分继续按原书顺序推进。第 10 章实践阅读 AT&T/Intel 风格反汇编。第 11 章实践检查位运算、小端组装和地址计算。第 12 章实践用错误输入路径训练提前返回和诊断。第 13 章实践解释 System V ABI 中的符号、`binding`、`type`、`section index`。第 14 章实践观察 `name mangling`、`extern"C` 和优化影响。第 15 章实践用 benchmark smoke 明确测量边界。第 16 章实践形成最终研究报告和回归检查。

每章正文都只保留 3 到 4 个大节：先说明对应原书问题，再从一个具体特例推出本章工程规律，然后给代码任务和命令，最后给验收标准和研究记录。完整源码仍放在附录中，方便读者把章节片段和真实工程对上。

目录

常见缩写和术语先导	iii
本卷结构	v
第一部分 C++ 程序如何成为进程	
第 0 章实践：学习地图与证据链	2
一、对应原书问题：主线不是名词表	2
二、从特例推到规律：文件名不是证据	2
三、实践任务：建立项目总地图	2
四、验收：能说明边界才算会用地图	3
第 1 章实践：程序如何从源码变成进程	4
一、对应原书问题：从源码到产物身份	4
二、从特例推到规律：magic 正确不等于解析安全	4
三、实践任务：编译、检查、交叉验证	4
四、验收：报告必须能复查	5
第 5 章实践：编译器 IR、优化 pass 与汇编证据链	6
一、对应原书问题：优化不是一句 Release 更快	6
二、从特例推到规律：函数可能消失	6
三、实践任务：比较两个构建产物	6
四、验收：优化结论必须降级	6
第 2 章实践：CPU 如何执行指令	8
一、对应原书问题：从 .text 到指令	8
二、从特例推到规律：地址不是一种	8
三、实践任务：定位一段机器码	8
四、验收：指令级分析不能跳步	8
第 3 章实践：数据表示、内存、指针和对象布局	10
一、对应原书问题：字段不是自动长出来的	10
二、从特例推到规律：小端读取和范围检查	10
三、实践任务：给读取函数补证据	10
四、验收：内存解释要可防错	10
第 4 章实践：Linux、工具链、调试器和学习 workflow	12
一、对应原书问题：工具不是截图生成器	12
二、从特例推到规律：一个工具输出不能独占真相	12
三、实践任务：建立固定 workflow	12
四、验收：workflow 要能给别人复跑	12
第二部分 x86-64、ABI 与可信测量	
第 10 章实践：x86-64 汇编语法总览	15
一、对应原书问题：反汇编是证据，不是装饰	15
二、从特例推到规律：语法风格会改变阅读方向	15
三、实践任务：标注五条指令	15
四、验收：能读出最小语义	15
第 11 章实践：整数指令、位运算和地址计算	16
一、对应原书问题：整数不是只有加减乘除	16
二、从特例推到规律：一个字节可以装两个字段	16
三、实践任务：解释三段整数代码	16
四、验收：数字要带语义	16
第 12 章实践：控制流、跳转、循环、调用与 switch	17
一、对应原书问题：提前返回也是控制流设计	17
二、从特例推到规律：坏输入路径要比好输入更清楚	17
三、实践任务：画出 inspect_elf 的路径	17
四、验收：错误路径不能含糊	17
第 13 章实践：System V AMD64 ABI	19
一、对应原书问题：ABI 不只是在说寄存器	19
二、从特例推到规律：源码函数不一定有同名符号	19
三、实践任务：保留一个 ABI 观察点	19

四、验收：ABI 结论要有层级	19
第 14 章实践：C++ 与汇编互操作	21
一、对应原书问题：源码名字不是链接名字	21
二、从特例推到规律：同一个函数有多种名字	21
三、实践任务：比较 C++ 和 extern"C"	21
四、验收：互操作要讲清边界	21
第 15 章实践：可信性能测量基础	23
一、对应原书问题：benchmark 不是时间数字	23
二、从特例推到规律：计时边界会污染结论	23
三、实践任务：运行 benchmark 并解释限制	23
四、验收：性能结论必须可降级	23
第 16 章实践：Benchmark 设计、输入、热路径和报告	25
一、对应原书问题：实验系统要能长期维护	25
二、从特例推到规律：快得离谱先怀疑实验	25
三、实践任务：最终研究项目目录	25
四、验收：第一册实践闭环	25
完整源码清单	27
一、构建入口	27
二、公共合同	29
三、实现和 CLI	35
四、测试和 Benchmark	45
术语表	49

第零部分

C++ 程序如何成为进程

第 0 章实践：学习地图与证据链

本章对应第一册第 0 章“学习地图：把程序、机器、系统和性能连成一条主线”。正文讲的是心智模型：不要把 C++、编译器、CPU、操作系统和性能测量分成互不相干的课。本章要把这个心智模型落成实践卷的总地图。最终你要能回答一个朴素问题：给定一个自己编译出的可执行文件，哪些结论来自文件字节，哪些结论来自工具解释，哪些结论来自运行时观察，哪些结论还只是猜测。

一、对应原书问题：主线不是名词表

如果只说“源码经过编译变成机器码，然后 CPU 执行”，读者很容易以为学习路线就是背名词。实践卷从一开始就要求把名词绑定到证据。源码文件是输入，编译命令是转换规则，ELF 文件是二进制产物，section 和 symbol 是文件里的结构化证据，loader 与进程是运行时事实，benchmark 是在特定环境下观察到的数字。

本卷贯穿项目 **Executable Evidence Kit**（可执行文件证据工具箱）的第一份交付不是代码，而是一张证据链表。表中至少要有五列：对象、能观察到的字段、观察命令、代码中的对应结构、当前限制。例如 `ELFheader` 的字段可以由 `cpu1ee_cli` 和 `readelf-h` 同时观察；`.text` 的 offset 和 size 可以由 section table 给出；真正装载后的虚拟地址则不能只靠文件 offset 推断。

核心思想

实践卷按原书章节走，但所有章节都围绕同一个工程对象。这样做的目的不是重复讲一遍概念，而是让每个概念都能落在同一份二进制证据上。

二、从特例推到规律：文件名不是证据

先看一个特例。你编译出 `hello`，随后重新编译一次，文件名仍然叫 `hello`。如果报告只写“我分析了 `hello`”，这份报告就没有固定分析对象，因为第二次构建已经覆盖了第一次构建。再看另一个特例：同一个源码用 Debug 和 Release 构建，文件名也许只差一个目录，但 section size、symbol 数量、反汇编和运行时间都可能不同。

由这两个特例可以推出第一条规律：实践报告必须绑定字节身份，而不是绑定口头描述。本卷用 `read_binary_file` 读入字节，用 `checksum_bytes` 计算摘要，用 byte size 记录长度。checksum 不是密码学证明，它的作用是把“这一次输入”固定下来，避免报告描述错对象。

第二条规律是区分文件事实和运行事实。ELF header、section table、symbol table 属于文件事实；进程地址空间、动态链接器状态、perf 事件属于运行事实。文件事实可以离线读取，运行事实必须在程序执行时观察。把这两类混在一起，就会把 section offset 误当成进程地址，或者把一次 benchmark 数字误当成二进制结构结论。

三、实践任务：建立项目总地图

先完成一次完整构建，确认后续章节使用的是同一份工程：

```
1 cmake -S books/cpu-volume-1-practice \
2     -B books/cpu-volume-1-practice/build/executable-evidence-debug \
3     -DCMAKE_BUILD_TYPE=Debug
4 cmake --build books/cpu-volume-1-practice/build/executable-evidence-debug
5 ctest --test-dir books/cpu-volume-1-practice/build/executable-evidence-debug \
6     --output-on-failure
```

然后阅读三个入口文件：`include/cpu1/executable_evidence.hpp`、`src/executable_evidence.cpp`、`src/main.cpp`。不要急着改代码，先写一张表，把结构体和概念对应起来：`ElfReport` 对应一次检查报告，`SectionSummary` 对应 section 摘要，`SymbolSummary` 对应符号摘要，`InspectionConfig` 对应解析边界。

本章交付物是 `reports/ch00-evidence-map.md`。报告中必须列出至少 8 个证据点：`source name`、`byte size`、`checksum`、`ELF magic`、`machine`、`entry address`、`.text section`、`symbol`、`diagnostic`、`benchmark smoke` 可以任选其八，但每一项都要写出它来自文件、工具、测试还是运行。

四、验收：能说明边界才算会用地图

本章验收不看你画得漂不漂亮，只看证据边界是否清楚。通过标准如下：

1. 能运行 `ctest`，并说明测试通过只证明当前合同，没有证明所有 ELF 都支持。
2. 能指出 `checksum` 固定的是输入字节，不是源码语义，也不是程序运行行为。
3. 能说明 `entryaddress` 不是 `main` 的地址。
4. 能区分 section 的 `file offset` 和进程中的 `virtual address`。
5. 能写出本卷第一版不支持的内容：32 位 ELF、大端 ELF、PE/Mach-O、DWARF、动态链接装载后地址。

如果这些边界讲不清，后续章节很容易变成“看见一个字段就下结论”。第 0 章实践的目就是把这种风险提前挡住。

第 1 章实践：程序如何从源码变成进程

本章对应第一册第 1 章“程序如何从源码变成正在运行的进程”。正文讲编译、链接、装载和进程，本章把这条路线压缩成一个可验证任务：编译一个最小 C++ 程序，再用 `cpu1ee_cli` 和系统工具证明它确实变成了某个 ELF64 小端二进制。

一、对应原书问题：从源码到产物身份

先写一个最小程序：

```
1 int add_i32(int a, int b) {
2     return a + b;
3 }
4
5 int main() {
6     return add_i32(20, 22) == 42 ? 0 : 1;
7 }
```

这段代码本身不能被 CPU 直接执行。实践要补上的不是一句“编译器会处理”，而是完整证据：用什么命令编译，产物有多少字节，magic 是否为 ELF，machine 是否是 x86-64，entry address 是多少，哪些 section 出现，工具输出是否能复查同一字段。

二、从特例推到规律：magic 正确不等于解析安全

最小反例是只有四个 magic 字节的文件：`0x7F'E'L'F'`。它看起来像 ELF，但长度不足 ELF64 header。若解析器看到 magic 就继续读 machine 和 entry，就会越界读取。第二个反例是 ELF32 或大端 ELF。magic 仍然正确，但字段宽度和字节序不符合本卷第一版合同。

由此推出解析顺序：先读字节，计算 checksum；再检查 magic；再检查 header 长度；再检查 class 和 endianness；最后才读取 object type、machine 和 entry address。这个顺序在 `inspect_elf` 里表现为一组提前返回。提前返回不是偷懒，而是在输入合同不满足时停止解释。

三、实践任务：编译、检查、交叉验证

把上面的程序保存到临时目录后执行：

```
1 g++ -std=c++20 -O0 -g hello.cpp -o hello-debug
2 g++ -std=c++20 -O2 hello.cpp -o hello-release
3
4 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
   ↵ executable_evidence/cpu1ee_cli \
5 ./hello-debug
6 readelf -h ./hello-debug
```

记录 `cpu1ee_cli` 输出中的 `bytes`、`checksum`、`machine`、`entry`、`sections` 和 `symbols`。再从 `readelf-h` 中找到 `Class`、`Data`、`Machine`、`Entry point address`。两个工具不需要输出格式一致，但字段含义必须能对应。

本章还要补一组坏输入测试思路。用空文件、四字节 magic 文件、把 fixture 的 class 改成 ELF32 的输入分别检查 diagnostic。源码里已经有

`RejectsNonElfMagic`、`RejectsShortElfHeader`、`RejectsUnsupportedClassOrEndian`，阅读它们并写出每个测试对应哪个失败特例。

四、验收：报告必须能复查

本章交付物是 `reports/ch01-program-to-process.md`。通过标准如下：

1. 报告写出编译命令，不只写“我编译了程序”。
2. 报告保存 Debug 和 Release 两个产物的 byte size 与 checksum。
3. 至少用 `readelf-h` 交叉验证 machine 和 entry 两个字段。
4. 能解释为什么 entry address 不等于 `main`，并说明启动代码会先运行。
5. 能把三个坏输入和三个 diagnostic 对上。

完成本章后，读者不只是“知道程序会变成进程”，而是能拿出一个具体二进制，说明它从哪里来、是什么格式、当前解析器能证明什么。

第 5 章实践：编译器 IR、优化 pass 与汇编证据链

本章对应第一册第 5 章“编译器 IR、优化 pass 与汇编证据链”。实践目标不是要求你实现编译器，而是训练一条判断路径：源码改动或优化等级改变后，二进制证据是否真的变化，变化出现在 section、symbol、反汇编还是 benchmark 数字里。

一、对应原书问题：优化不是一句 Release 更快

同一份 `hello.cpp` 用 `-O0` 和 `-O2` 编译，得到的二进制可能大小不同，符号数量不同，函数是否保留独立符号也不同。初学者常见错误是直接说“Release 更快”，但没有证明当前跑的就是 Release 产物，也没有证明热函数还以独立形态存在。

实践卷要求先固定产物身份，再讨论优化。固定身份靠 `byte size` 和 `checksum`；观察结构靠 `section` 与 `symbol`；观察机器码靠 `objdump-d`；讨论运行时间前，还要知道 benchmark 计时边界里有没有 I/O、打印、构造输入等无关成本。

二、从特例推到规律：函数可能消失

特例一：`intadd_i32(int,int)` 在 `-O0` 下通常会保留一个可见函数体，反汇编中能找单独标签。特例二：在 `-O2` 下，如果 `add_i32` 很小且只被 `main` 调用，编译器可能把它 `inline` 到调用点，符号表中不再以你期待的方式出现。此时不能说“解析器没找到所以函数不存在于源码”，只能说“当前构建产物里没有以独立符号形式保留它”。

由此推出规律：源码层概念和二进制层证据不是一一等价。编译器 IR 与优化 pass 会改变函数边界、常量表达、循环形态和调用结构。实践报告必须写清楚观察层级：源码函数、IR 中的函数、目标文件符号、最终可执行文件符号、反汇编标签，分别可能不同。

三、实践任务：比较两个构建产物

执行两组构建，并保存输出：

```
1 g++ -std=c++20 -O0 -g hello.cpp -o hello-o0
2 g++ -std=c++20 -O2 hello.cpp -o hello-o2
3
4 cpu1ee=books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
   ↵ executable_evidence/cpu1ee_cli
5 $cpu1ee ./hello-o0 > reports/hello-o0.report
6 $cpu1ee ./hello-o2 > reports/hello-o2.report
7 $cpu1ee ./hello-o0 --sections-csv > results/hello-o0.sections.csv
8 $cpu1ee ./hello-o2 --sections-csv > results/hello-o2.sections.csv
9
10 objdump -d ./hello-o0 > reports/hello-o0.disasm
11 objdump -d ./hello-o2 > reports/hello-o2.disasm
```

比较时不要只看文件大小。至少检查三类证据：`checksum` 是否不同，`.text` 的 `size` 是否变化，反汇编里 `add_i32` 是否保留独立标签。若符号没有出现，继续用 `nm-C` 或 `objdump-t` 辅助观察，并在报告中写明“未找到独立符号”的范围。

四、验收：优化结论必须降级

本章交付物是 `reports/ch05-optimization-evidence.md`。通过标准如下：

1. 报告同时包含 `-O0` 和 `-O2` 的构建命令、checksum 和 byte size。
2. 至少比较一个 section 字段和一个 symbol 或反汇编证据。
3. 如果函数被 inline 或符号被优化掉，报告必须写成“当前产物未以独立符号保留”，不能写成“函数不存在”。
4. 能解释为什么 benchmark 数字必须绑定当前 checksum。
5. 能指出本卷解析器没有解析 IR，因此 IR 结论需要来自编译器输出或其他工具，而不是从 ELF 字段硬推。

这章完成后，读者应该形成一个习惯：优化相关判断先问“我观察的是哪一层”，再问“这层证据能支持多强的结论”。

第 2 章实践：CPU 如何执行指令

本章对应第一册第 2 章“CPU 如何执行指令”。正文讲 `fetch`、`decode`、`execute`、寄存器、内存访问、分支和乱序执行。本章不试图模拟 CPU，而是做一个可操作的小目标：从 ELF 的 `.text` section 找到机器码区域，再用反汇编把字节、指令和函数符号连起来。

一、对应原书问题：从 `.text` 到指令

第一册正文会反复提醒：CPU 执行的是机器指令，不是 C++ 源码行。实践中最容易断开的地方是中间证据。你看到 `add_i32` 的源码，却不知道它在二进制里对应哪个符号；你看到 `objdump` 输出，却不知道这些指令来自文件的哪个区域。

本章要把三件事连起来：`cpulee_cli--sections-csv` 输出 `.text` 的 `offset` 和 `size`；`objdump-d` 输出反汇编；`symbol table` 里的函数名指向某个 section `index` 和 `value`。三者合起来，才形成“这个函数的机器指令在当前二进制中如何呈现”的证据链。

二、从特例推到规律：地址不是一种

特例一：section CSV 里的 `offset` 是文件偏移，表示从文件开头数多少字节能到达该 section 内容。特例二：反汇编里函数标签旁边的地址通常是虚拟地址或链接地址，不是文件偏移。特例三：运行时进程启用 PIE 和 ASLR 后，实际装载地址还可能再变化。

由此推出规律：看到一个十六进制数，先问它属于哪种地址空间。文件 `offset`、ELF `virtual address`、链接地址、运行时虚拟地址不能混用。本卷解析器第一版只报告 section 的 `file offset`、`section address`、`symbol value`，不负责把运行时地址反推回文件位置。

三、实践任务：定位一段机器码

先输出 section 表：

```
1 cpulee=books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
  ↵ executable_evidence/cpulee_cli
2 $cpulee ./hello-o0 --sections-csv | tee results/hello-o0.sections.csv
3 objdump -d ./hello-o0 | tee reports/hello-o0.disasm
```

在 CSV 中找到 `.text` 行，记录 `address`、`offset`、`size`。再在反汇编中找到 `add_i32` 或 `main`。如果找不到 `add_i32`，回到第 5 章实践的优化解释：当前构建可能 `inline` 了它，也可能符号不可见。为了观察指令，可以临时给函数加 `__attribute__((noinline))`，但报告必须写明这是为了保留观察点。

阅读源码中的 `SectionSummary` 和 `SymbolSummary`，回答：`section.address`、`section.offset`、`symbol.value` 分别表达什么？为什么 `symbol.value` 不能直接传给 `std::span` 当文件偏移？

四、验收：指令级分析不能跳步

本章交付物是 `reports/ch02-instruction-evidence.md`。通过标准如下：

1. 报告列出 `.text` 的 `address`、`offset`、`size`。
2. 报告贴出一小段反汇编，并说明它来自哪个函数或标签。
3. 能解释至少一条指令的操作数是寄存器还是内存访问。

4. 能说明 file offset、section address、symbol value 的区别。
5. 不把 CPU 微架构行为当成已经证明的事实；本章只证明文件和反汇编层面的指令证据。

完成本章后，读者应该能从“源码函数”走到“当前二进制里可观察到的机器指令”，而不是凭直觉想象 CPU 在执行什么。

第 3 章实践：数据表示、内存、指针和对象布局

本章对应第一册第 3 章“数据表示、内存、指针和对象布局”。正文讲字节、类型、对象表示、对齐、指针和布局。本章把这些概念落到 ELF 解析器的核心实现：所有字段最终都来自一段字节，解析器必须按小端规则组合整数，并且每次读取前都要证明范围存在。

一、对应原书问题：字段不是自动长出来的

ELF header 里的 `machine` 是一个 16 位整数，`entry` 是一个 64 位整数，section header 里还有 `offset`、`size`、`flags` 等字段。文件中没有“整数对象”，只有连续字节。解析器把这些字节解释成整数，靠的是格式合同和字节序规则。

源码里的 `read_u16_le`、`read_u32_le`、`read_u64_le` 就是本章主角。它们不依赖把字节强转成结构体指针，而是逐字节组合。这种写法繁琐一点，但能清楚处理未对齐、对象生命周期、别名和越界问题，适合教学和可移植解析器。

二、从特例推到规律：小端读取和范围检查

先看特例：字节序列 78563412 按 little-endian 解释为 0x12345678，按 big-endian 则是 0x78563412。如果不先检查 ELF ident 的 endianness，就无法知道同一段字节该怎样解释。

再看范围特例：输入长度为 10，想从 offset 8 读取 8 字节。offset 本身没有越界，但 offset 加 size 已经超过输入末尾。源码里的 `has_range` 用 `offset <= byte_count && size <= byte_count - offset` 避免加法溢出和越界读取。这比直接判断 `offset + size <= byte_count` 更稳。

由这两个特例推出本章规律：底层解析先证明“能读”，再证明“该怎样读”，最后才讨论“读出来代表什么”。顺序一乱，就会把未定义行为、越界和格式错误混成一个难排查的问题。

三、实践任务：给读取函数补证据

阅读 `src/executable_evidence.cpp` 中的四个函数：`has_range`、`read_u16_le`、`read_u32_le`、`read_u64_le`。然后在测试报告中写出三个样例：

1. 0100 读成 1；
2. 78563412 读成 0x12345678；
3. offset 接近输入末尾时必须拒绝读取。

本卷当前没有把这些私有函数直接暴露给测试，这是一个工程选择：对外合同测试通过 synthetic ELF 间接覆盖它们。作为本章加分任务，可以讨论是否应该把小端读取抽到内部可测试模块。若抽出，必须避免扩大公开 API；测试可以放在同一个 translation unit 或通过小型内部 helper 完成。

四、验收：内存解释要可防错

本章交付物是 `reports/ch03-bytes-layout.md`。通过标准如下：

1. 能用自己的话解释 little-endian，不只写“小端就是反过来”。
2. 能说明为什么本项目不把 ELF header 直接 `reinterpret_cast` 成 C++ struct。
3. 能解释 `has_range` 为什么不用简单的 `offset + size` 判断。
4. 能把 section 的 offset 和 size 看成“文件字节范围”，而不是抽象名词。

5. 能指出 padding、alignment、对象生命周期这些 C++ 规则和文件格式解析之间的边界。

这章学完后，读者再看到 section、symbol、entry 时，应该能自然追问：这个字段从哪些字节来，按什么规则解释，读之前怎样证明安全。

第 4 章实践：Linux、工具链、调试器和学习工作流

本章对应第一册第 4 章“Linux、工具链、调试器和学习工作流”。正文讲命令行、编译器、调试器、`readelf`、`objdump`、`perf` 和可复现实验。本章把这些工具固定成一个日常工作流：先构建，再测试，再用工具交叉验证，最后把证据写入报告。

一、对应原书问题：工具不是截图生成器

初学者常见做法是运行一堆命令，把终端输出截图放进文档，然后写“如图所示”。这不够。工具输出必须回答具体问题：`readelf-h` 验证 ELF header，`readelf-S` 验证 section table，`readelf-s` 或 `objdump-t` 验证 symbol，`objdump-d` 验证反汇编，`ctest` 验证代码合同。

本章实践目标是建立一套可重复命令，而不是临时点几下。每个命令都要写入报告，必要时把输出保存到 `reports/` 或 `results/`，这样别人才能复跑。

二、从特例推到规律：一个工具输出不能独占真相

特例：`cpu1ee_cli` 和 `readelf` 都能报告 machine。如果两者一致，说明我们对该字段的解析更可信；如果不一致，不能立刻说系统工具错了，更可能是我们的解析器、输入文件或字段理解有问题。另一个特例是 `objdump-d` 能看到反汇编，但它不能替你证明 benchmark 可信。

由此推出规律：工具要按问题组合使用。结构问题用 `readelf` 和本卷解析器交叉验证；指令问题用 `objdump`；运行问题用 `perf` 或 benchmark；正确性问题用 GTest/GMock；构建问题用 CMake 和 `ctest`。工具越多不代表越严谨，能互相验证同一个事实才严谨。

三、实践任务：建立固定工作流

本章要求你把下面命令保存成自己的实验记录模板：

```
1 cmake -S books/cpu-volume-1-practice \
2     -B books/cpu-volume-1-practice/build/executable-evidence-debug \
3     -DCMAKE_BUILD_TYPE=Debug
4 cmake --build books/cpu-volume-1-practice/build/executable-evidence-debug
5 ctest --test-dir books/cpu-volume-1-practice/build/executable-evidence-debug \
6     --output-on-failure
7
8 cpu1ee=books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
9     ↵ executable_evidence/cpu1ee_cli
9 $cpu1ee "$binary" > "reports/$(basename "$binary").report"
10 $cpu1ee "$binary" --sections-csv > "results/$(basename "$binary").sections.csv"
11 readelf -h "$binary" > "reports/$(basename "$binary").readelf-header.txt"
12 readelf -S "$binary" > "reports/$(basename "$binary").readelf-sections.txt"
13 objdump -d "$binary" > "reports/$(basename "$binary").disasm.txt"
```

然后做一次字段核对：从 `cpu1ee_cli` 报告中取 machine 和 entry，从 `readelf header` 中找同一字段；从 section CSV 中取 `.text` 的 offset 和 size，从 `readelf-S` 中找对应行。若格式不同，报告里写字段名如何对应。

四、验收：工作流要能给别人复跑

本章交付物是 `reports/ch04-linux-tool-workflow.md`。通过标准如下：

1. 报告列出构建、测试、CLI、`readelf`、`objdump` 的完整命令。
2. 至少有三个字段被两个独立工具交叉验证。
3. 能说明 `ctest` 验证的是代码行为，不验证真实二进制分析报告是否写得正确。
4. 若当前环境缺少 `perf` 或权限不足，报告写明限制，不把缺失数据伪装成结论。
5. 所有输出文件放在 `reports/` 或 `results/`，命名能看出对应的输入二进制。

这章的重点不是记住每个命令选项，而是形成一个严谨习惯：每次观察都有问题、有命令、有输出、有对照、有边界。

第零部分

x86-64、ABI 与可信测量

第 10 章实践：x86-64 汇编语法总览

本章对应第一册第 10 章“x86-64 汇编语法总览”。正文讲寄存器、立即数、内存操作数、AT&T 与 Intel 风格差异。本章要做的是把反汇编从“看不懂的一堆字符”变成可核对证据：每条指令至少能分清助记符、源操作数、目的操作数、寄存器还是内存。

一、对应原书问题：反汇编是证据，不是装饰

很多报告会贴一段 `objdump-d` 输出，然后直接说“可以看到汇编”。这不够。本章要求你选择一个很小函数，例如 `add_i32` 或 `main` 中的一段，逐行标注：指令地址、机器码字节、助记符、操作数、操作数含义。

这一步要和本卷解析器连接起来。先用 `--sections-csv` 找到 `.text`，再用反汇编观察 `.text` 中的代码。这样汇编不是孤立文本，而是 ELF section 中字节的解释。

二、从特例推到规律：语法风格会改变阅读方向

特例：AT&T 风格通常写作 `movl%edi,-4(%rbp)`，源操作数在前，目的操作数在后；Intel 风格常写作 `movDWORDPTR[rbp-4],edi`，目的操作数在前，源操作数在后。两者描述的可能是同一条机器指令，但文本顺序不同。

由此推出规律：读汇编时先确认语法风格。不要看到两个操作数就凭直觉猜方向。对于本章报告，建议固定使用 `objdump-d-Mintel` 或默认 AT&T 风格之一，并在报告开头写明。若对照两种风格，必须说明同一条指令如何对应。

三、实践任务：标注五条指令

执行：

```
1 objdump -d ./hello-o0 > reports/hello-o0.att.txt
2 objdump -d -Mintel ./hello-o0 > reports/hello-o0.intel.txt
```

从 `main` 或 `add_i32` 中选择五条连续指令，并按四个字段标注。第一是地址，例如 `401126`，它是反汇编视角的代码位置。第二是机器码字节，例如 `55`，它是 CPU 实际解码的输入。第三是助记符，例如 `push`，它描述操作种类。第四是操作数，例如 `rbp`，它说明本条指令访问的是寄存器、立即数还是内存。

如果编译器生成的指令和书中示例不同，不要强行改报告。写明编译器版本、优化等级和构建命令，然后解释当前产物中实际出现的指令。

四、验收：能读出最小语义

本章交付物是 `reports/ch10-assembly-syntax.md`。通过标准如下：

1. 报告明确使用 AT&T 还是 Intel 风格。
2. 至少标注五条指令，每条都分出助记符和操作数。
3. 至少说明一条寄存器操作和一条内存操作。
4. 能把反汇编片段和 `.text` section 联系起来。
5. 不根据助记符名称过度推断性能；本章只训练语法和证据定位。

完成本章后，读者应该能把反汇编当成可阅读证据，而不是只把它当作“底层感很强”的截图。

第 11 章实践：整数指令、位运算和地址计算

本章对应第一册第 11 章“整数指令、位运算和地址计算”。实践目标有两条：一是读懂反汇编里的整数加法、位移、按位与或和地址计算；二是回到解析器源码，理解小端整数读取本身就是一组位运算。

一、对应原书问题：整数不是只有加减乘除

ELF 解析器中到处都是整数：offset、size、address、flags、type、binding、section index。它们有的表示数量，有的表示地址，有的是位标志，有的是枚举值。若把所有整数都当普通十进制数看，就看不出格式结构。

源码中的 `read_u32_le` 展示了最基础的位运算：取一个字节，转换成更宽整数，再左移 `i*8` 位，最后按位或到结果里。符号解析中的 `info&0x0F` 和 `info>>4` 则展示了同一个字节里拆出 type 和 binding 的方法。

二、从特例推到规律：一个字节可以装两个字段

特例：ELF symbol 的 `st_info` 字节高 4 位表示 binding，低 4 位表示 type。若 `info` 是 `0x12`，低 4 位是 2，高 4 位是 1。这不是两个字节，也不是字符串，而是一个字节里的两个小字段。

由此推出规律：二进制格式常把多个小字段压在同一个整数里。读取只是第一步，拆字段还需要 mask 和 shift。mask 用来保留关心的位，shift 用来把高位字段移动到从 0 开始的数值位置。理解这一点后，section flags、权限位、CPU flags 都会更容易读。

三、实践任务：解释三段整数代码

阅读 `src/executable_evidence.cpp` 中三段代码：

1. `read_u64_le` 如何把 8 个字节合成一个 `uint64_t`；
2. `has_range` 如何避免越界和整数溢出；
3. `summarize_symbols` 如何从 `info` 拆出 type 和 binding。

再写一个小表，用具体数字解释 `info` 拆分：

```
1 std::uint8_t info = 0x12;
2 std::uint8_t type = info & 0x0F;      // 2
3 std::uint8_t binding = info >> 4U;    // 1
```

最后在反汇编中找一条地址计算或整数操作指令，例如 `add`、`lea`、`and`、`shl`、`shr`。说明它的输入、输出和是否访问内存。

四、验收：数字要带语义

本章交付物是 `reports/ch11-integer-instructions.md`。通过标准如下：

1. 能解释 `read_u64_le` 的每次移位为什么是 8 的倍数。
2. 能解释 `info&0x0F` 和 `info>>4U` 分别保留哪部分。
3. 能把至少一个 section flags 或 symbol type 数值翻译成人能读的含义。
4. 能在反汇编中标注一条整数或地址计算指令。
5. 不把所有整数都叫地址；报告要区分枚举、大小、偏移、虚拟地址和位标志。

本章的核心训练是“数字带上下文”。没有上下文，二进制字段只是一串数；有了格式和位运算，数字才变成证据。

第 12 章实践：控制流、跳转、循环、调用与 switch

本章对应第一册第 12 章“控制流：跳转、循环、调用与 switch”。实践卷把控制流分成两层：机器码中的控制流，以及解析器源码中的错误路径控制流。本章重点训练后者，因为可靠工具首先要在坏输入面前走对路径。

一、对应原书问题：提前返回也是控制流设计

控制流不只是汇编里的 `jmp`、`call`、`ret`。在解析器里，`inspect_elf` 面对坏输入时必须选择是否继续。`magic` 不对，后续所有 ELF 字段都没有意义；`header` 太短，继续读取会越界；`class` 或 `endianness` 不支持，继续解释会得到错误字段。

因此本章把提前返回当成工程控制流来分析。每个提前返回都对应一个无法满足的前置条件，并且必须留下 `diagnostic`。没有 `diagnostic` 的提前返回会让用户不知道失败原因；没有提前返回的坏输入会污染后续证据。

二、从特例推到规律：坏输入路径要比好输入更清楚

特例一：输入是普通文本文件。正确路径是设置 `valid_magic=false`，追加 `input_does_not_start_with_elf_magic`，返回。特例二：输入有 `magic` 但只有 16 字节。正确路径是承认 `magic` 正确，但追加 `input_is_shorter_than_elf64_header`，返回。特例三：输入是 ELF32。正确路径是记录只支持 ELF64 小端，而不是尝试按 ELF64 读取。

由此推出规律：解析器的控制流应该由格式前置条件驱动。每通过一个前置条件，才能进入下一层读取。测试也要按这些条件组织，而不是只测试一个“正常文件能跑”。

三、实践任务：画出 `inspect_elf` 的路径

阅读 `inspect_elf`，画出如下状态图的文字版：

1. 创建 `report`，记录 `source`、`byte size`、`checksum`；
2. 检查 `magic`，失败则 `diagnostic` 后返回；
3. 检查 `header` 长度，失败则 `diagnostic` 后返回；
4. 检查 `class` 和 `endianness`，失败则 `diagnostic` 后返回；
5. 读取 `header` 字段；
6. 解析 `section`；
7. 解析 `symbol`；
8. 返回完整 `report`。

然后把测试文件中的三个坏输入测试与路径节点对应起来：`RejectsNonElfMagic` 对应第 2 步，`RejectsShortElfHeader` 对应第 3 步，`RejectsUnsupportedClassOrEndian` 对应第 4 步。最后在反汇编中观察一个真实函数调用，说明源码层调用、汇编层 `call` 和解析器控制流分析是不同层级的问题。

四、验收：错误路径不能含糊

本章交付物是 `reports/ch12-control-flow.md`。通过标准如下：

1. 能画出 `inspect_elf` 的主要控制路径。
2. 每个提前返回都写明触发条件和 `diagnostic`。
3. 能解释为什么坏 `magic` 后不能继续读 `header`。

4. 能说明测试覆盖了哪些错误路径，还没有覆盖哪些路径，例如 section table 越界。
5. 能区分程序源码控制流、反汇编控制流和解析器错误处理控制流。

这章的结论很实用：一个工具是否可靠，往往不是看正常输入有多快，而是看坏输入是否被清楚、稳定、可测试地处理。

第 13 章实践：System V AMD64 ABI

本章对应第一册第 13 章“System V AMD64 ABI”。ABI 是 **Application Binary Interface**（应用二进制接口），它规定二进制层面的函数调用、寄存器使用、栈对齐、符号和链接约定。本章不要求一次掌握全部 ABI，而是从符号表和一个未内联函数开始，把源码函数连接到二进制可见边界。

一、对应原书问题：ABI 不只是在说寄存器

很多人学 ABI 时只记住“前几个整数参数放在 `rdi`、`rsi`、`rdx`...”。这当然重要，但 ABI 还体现在符号、调用边界、栈、返回值、链接器可见性等地方。实践卷的入口是 `symbol table`，因为它能告诉你某个函数或对象是否在当前二进制中作为符号出现。

`SymbolSummary` 记录 `name`、`section name`、`type`、`binding`、`section index`、`value` 和 `size`。它不是完整 ABI 模型，但足以回答几个关键问题：这个名字是否可见，它属于哪个 `section`，它是函数还是对象，它的地址值是什么，它是否可能被优化、strip 或 `name mangling` 改变。

二、从特例推到规律：源码函数不一定有同名符号

特例一：C++ 函数 `intadd_i32(int,int)` 可能在符号表中出现为经过 `name mangling` 的名字，而不是源代码中的简单名字。特例二：函数被 `inline` 后，最终可执行文件中可能没有独立函数符号。特例三：用 `extern"C` 声明的函数名字通常更接近源码名字，但仍受可见性、链接和优化影响。

由此推出规律：ABI 观察要同时看源码声明、编译命令、符号表、反汇编和优化等级。符号表不能单独证明所有调用约定，但它能告诉我们当前二进制暴露了哪些边界。想观察寄存器传参，还需要反汇编或调试器。

三、实践任务：保留一个 ABI 观察点

写一个不容易被内联的小函数：

```
1 extern "C" __attribute__((noinline))
2 int abi_add_i32(int a, int b) {
3     return a + b;
4 }
```

编译后观察：

```
1 g++ -std=c++20 -O2 -g abi_sample.cpp -o abi-sample
2 cpulee=books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
   ↵ executable_evidence/cpulee_cli
3 $cpulee ./abi-sample > reports/abi-sample.report
4 readelf -s ./abi-sample > reports/abi-sample.symbols.txt
5 objdump -d -Intel ./abi-sample > reports/abi-sample.disasm.txt
```

在 `readelf-s` 中找 `abi_add_i32`，记录 `symbol value`、`size`、`type`、`binding`、`section index`。再在反汇编中找函数入口，观察参数是否从约定寄存器进入计算。若编译器仍然改变形态，报告要写实际观察结果，而不是强行套模板。

四、验收：ABI 结论要有层级

本章交付物是 `reports/ch13-system-v-abi.md`。通过标准如下：

1. 能解释 ABI 全称和它解决的问题。
2. 报告中至少有一个未内联函数的 symbol 证据。
3. 能说明 type、binding、section index、value、size 的含义。
4. 能用反汇编观察至少一个参数寄存器或返回寄存器。
5. 能说明本卷解析器没有完整验证调用约定，它只是提供符号和 section 证据的一部分。

这章完成后，读者应该把 ABI 看成一组可观察的二进制约约，而不是一张寄存器背诵表。

第 14 章实践：C++ 与汇编互操作

本章对应第一册第 14 章“C++ 与汇编互操作”。实践目标是理解 C++ 名字、符号名字、链接边界和反汇编之间的关系。你不需要一开始就写复杂汇编文件，但必须知道为什么 C++ 函数名到了二进制层可能变样，为什么 `extern"C` 常被用作互操作边界。

一、对应原书问题：源码名字不是链接名字

C++ 支持函数重载、命名空间、类成员函数等特性。为了让链接器区分这些函数，编译器会把源码名字编码成符号名字，这就是 **name mangling**（名字修饰）。例如两个同名但参数不同的函数在源码里都叫 `add`，链接层必须变成不同符号。

实践中，汇编、C、动态链接和工具脚本经常需要稳定符号名。此时可以用 `extern"C` 关闭 C++ 名字修饰，让函数以 C ABI 风格暴露。它不是让函数变成 C 语言实现，而是改变链接名和调用边界约定。

二、从特例推到规律：同一个函数有多种名字

特例一：源码中写 `intadd(int,int)`，符号表里可能看到类似 `_Z3addii` 的名字。特例二：用 `nm-C` 可以把修饰名 `demangle` 回更接近源码的形式。特例三：用 `extern"C"intadd(int,int)` 后，符号表里更可能直接出现 `add`。

由此推出规律：报告中必须说明自己使用的是源码名字、`mangled symbol name`，还是 `demangled display name`。把三者混用，会导致你以为函数消失、重复或链接失败。

三、实践任务：比较 C++ 和 `extern"C`

写两个函数：

```
1 int cpp_add(int a, int b) {
2     return a + b;
3 }
4
5 extern "C" int c_add(int a, int b) {
6     return a + b;
7 }
```

编译并观察：

```
1 g++ -std=c++20 -O0 -g interop.cpp -o interop
2 nm ./interop > reports/interop.nm.txt
3 nm -C ./interop > reports/interop.nm-demangled.txt
4 readelf -s ./interop > reports/interop.symbols.txt
5 objdump -d ./interop > reports/interop.disasm.txt
```

再用 `cpu1ee_cli` 输出 report，检查 symbol 数量上限是否影响观察。如果 report 中符号被 `max_symbols` 截断，说明当前工具为了报告稳定性做了上限控制；需要用 `readelf` 或调整配置继续观察。

四、验收：互操作要讲清边界

本章交付物是 `reports/ch14-cpp-assembly-interop.md`。通过标准如下：

1. 能解释 name mangling 和 demangling 的区别。
2. 至少比较一个普通 C++ 函数和一个 `extern"C` 函数的符号名。
3. 能说明符号名稳定为什么对汇编互操作有价值。
4. 能指出 `extern"C` 不等于关闭优化，也不等于改变函数体语言。
5. 能把 `cpu1ee_cli` 的 symbol 摘要和 `nm/readelf-s` 输出对应起来。

这章的收获是：C++ 与汇编互操作不是“在 C++ 里夹一点汇编”这么简单，它首先要求你知道链接器看到的名字和边界是什么。

第 15 章实践：可信性能测量基础

本章对应第一册第 15 章“可信性能测量基础”。正文讲时间源、warmup、重复测量、噪声、偏差、PMU 和报告结构。本章把这些原则落到当前项目的 Google Benchmark smoke：它只测解析 synthetic ELF 的路径，不测文件读取，不测打印，也不等于真实 workload 性能结论。

一、对应原书问题：benchmark 不是时间数字

一个 benchmark 输出了纳秒数，并不自动可信。先问四个问题：测的输入是什么，计时边界在哪里，正确性是否已由测试保证，当前二进制身份是否固定。本项目里，正确性主要由 GTest/GMock 固定，benchmark 只对解析路径做 smoke 检查。

这就是“测量基础”的核心：性能数字必须附着在实验协议上。没有协议，数字无法复查；没有正确性测试，速度没有意义；没有 checksum 和构建信息，数字无法绑定到具体产物。

二、从特例推到规律：计时边界会污染结论

特例一：如果 benchmark 循环里每次都构造 synthetic ELF，那么测到的是构造和分配成本，不只是解析成本。特例二：如果循环里打印 report，那么 I/O 会主导时间。特例三：如果 benchmark 读真实文件，磁盘缓存、文件系统和路径解析会进入计时边界。

由此推出规律：microbenchmark 要把 setup 放在计时边界外，把热路径放在计时边界内，并把这个边界写进报告。本项目 benchmark 选择 synthetic ELF，是为了让输入稳定、体积小、可重复。它不能代表所有真实 ELF 文件，但可以作为解析入口没有明显退化的 smoke。

三、实践任务：运行 benchmark 并解释限制

先运行完整测试：

```
1 cmake -S books/cpu-volume-1-practice \
2     -B books/cpu-volume-1-practice/build/executable-evidence-debug \
3     -DCMAKE_BUILD_TYPE=Debug
4 cmake --build books/cpu-volume-1-practice/build/executable-evidence-debug
5 ctest --test-dir books/cpu-volume-1-practice/build/executable-evidence-debug \
6     --output-on-failure
```

然后找到 benchmark 可执行文件并运行。若本地 Google Benchmark 输出格式不同，以实际构建目录为准：

```
1 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
   ↪ executable_evidence/cpu1ee_bench
```

阅读 `bench/executable_evidence_bench.cpp`，说明输入构造在哪里，计时循环在哪里，`benchmark::DoNotOptimize` 保护的是什么。报告中不要只贴结果数字，必须写清“这个 benchmark 没有测文件读取、没有测 CLI 参数解析、没有测打印、没有测真实大文件”。

四、验收：性能结论必须可降级

本章交付物是 `reports/ch15-measurement-foundations.md`。通过标准如下：

1. 报告记录构建类型、CPU 环境能获得的信息、benchmark 命令。
2. 写清 benchmark 的输入和计时边界。
3. 写清 GTest/GMock 与 benchmark 的分工：测试证明行为，benchmark 观察成本。
4. 至少列出三个没有被当前 benchmark 覆盖的成本。
5. 若结果波动或环境受限，结论要降级为 smoke，不写成确定性能排名。

这章完成后，读者应该不再把 benchmark 当作“跑一下看数字”，而是当作一份需要协议、边界和限制的工程证据。

第 16 章实践：Benchmark 设计、输入、热路径和报告

本章对应第一册第 16 章“Benchmark 设计：输入、热路径和报告”。这是第一册实践卷的收束章。你要把前面所有章节的材料合成一个最终研究项目：对一个自己编译的二进制，提交可复查的结构报告、工具交叉验证、符号和反汇编解释、benchmark smoke 与限制说明。

一、对应原书问题：实验系统要能长期维护

一次 benchmark 可以临时跑，benchmark suite 则要长期维护。长期维护需要固定输入、固定输出 schema、保留 raw data、记录环境和构建、区分正确性失败与性能波动。本卷第一版没有做完整性能平台，但已经具备最小骨架：CMake 构建、GTest/GMock 测试、CLI report、section CSV、Google Benchmark smoke 和人工研究报告。

最终项目要把这些部件连起来，而不是各自截图。报告里每个结论都要能追溯到命令、输出文件、源码或测试。

二、从特例推到规律：快得离谱先怀疑实验

特例一：某次 benchmark 快到几乎为零，可能是编译器删除了工作，也可能是输入太小、batch 不够、计时边界错了。特例二：一次 Release 比 Debug 慢，可能是环境噪声，也可能是优化改变了代码布局、分支或符号。特例三：不同机器上结果差很多，可能是 CPU、频率、内存、内核权限和后台负载不同。

由此推出规律：benchmark 报告先写实验协议，再写数字，再写解释。解释必须能降级：证据强时可以说“在本环境、本输入、本构建下观察到”；证据弱时只能说“需要进一步复核”。

三、实践任务：最终研究项目目录

建立如下目录结构：

```
1 reports/final/
2 results/final/
3 materials/final/
```

最终项目至少包含九类文件。build.md 写源码、编译命令、编译器版本和构建类型。binary.report 保存 cpu_lee_cli 的文本报告。sections.csv 保存 cpu_lee_cli--sections-csv 输出。readelf-header.txt 保存 readelf-h 输出。readelf-sections.txt 保存 readelf-S 输出。symbols.txt 保存 readelf-s 或 nm-C 输出。disasm.txt 保存 objdump-d 或 objdump-d-Mintel 输出。benchmark.txt 保存 benchmark smoke 输出和限制说明。final-report.md 从这些证据出发写研究结论。

最终报告要按“事实、解释、限制”三段写。事实只能来自命令输出和测试；解释要说明字段之间如何互相支撑；限制要写当前工具不支持什么、当前 benchmark 没测什么、当前环境可能影响什么。

四、验收：第一册实践闭环

最终项目通过标准如下：

1. 能从源码、构建命令、checksum 定位同一个二进制产物。

2. 能用本卷解析器和 `readelf` 交叉验证至少三个字段。
3. 能解释一个函数的符号、`section`、反汇编和优化影响。
4. 能运行 `GTest/GMock` 测试，并说明它们覆盖的合同。
5. 能运行 `benchmark smoke`，并清楚说明它没有测哪些成本。
6. 最终结论不越界：没有证据的地方写限制，不写成事实。

验收与评分标准

第一册实践卷的合格终点不是“代码能跑”，而是读者能拿一个真实构建产物，从字节、格式、工具、指令、ABI、测试和测量七个角度解释它。目录按原书章节组织，是为了让每个正文概念都在同一个工程里完成一次落地。

完整源码清单

本附录从 [books/cpu-volume-1-practice/labs/executable_evidence](#) 直接读入完整工程源码。读者不要只看正文里的片段；真正的能力来自完整构建、完整测试和完整报告。

一、构建入口

Listing 13.1 第一册实践卷顶层 CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume1Practice LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_1_PRACTICE_BUILD_LABS "Build CPU Volume 1 practice labs" ON)
10
11 if(CPU_VOLUME_1_PRACTICE_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/executable_evidence)
14 endif()
```

Listing 13.2 Executable Evidence Kit README.md

```
1 # Executable Evidence Kit
2
3 这是第一册实践卷的贯穿项目。它把“C++ 源码如何成为进程”落成一个可测试工具：读取 ↵
    ↵ ELF64 小端可执行文件，解析 header、section table、string table 和 symbol ↵
    ↵ table，输出稳定文本报告和 section CSV。
4
5 ## 阶段路线
6
7 1. 字节读取：用 `read_binary_file` 把文件内容读成 `std::vector<std::uint8_t>`，↵
    ↵ 并用 checksum 固定输入身份。
8 2. ELF header：检查 magic、class、endianness、machine 和 entry address，区分“不↵
    ↵ 是 ELF”“不是 ELF64”“不是小端”。
9 3. Section table：解析 `.text`、`.shstrtab`、`.symtab` 和 `.strtab`，把 section↵
    ↵ name、offset、size 和 flags 输出成 CSV。
10 4. Symbol table：解析符号名、绑定、类型、所在 section、地址和大小，把 ABI/链接↵
    ↵ 视角接回源码函数。
11 5. CLI 证据：让工具可以检查自己或其他二进制，输出 report 或 section CSV。
12 6. 测试与 benchmark：用 GTest/GMock 固定边界，用 Google Benchmark 对解析路径做 ↵
    ↵ smoke。
13
14 ## 构建
15
```

```

16 ```bash
17 cmake -S books/cpu-volume-1-practice \
18       -B books/cpu-volume-1-practice/build/executable-evidence-debug \
19       -DCMAKE_BUILD_TYPE=Debug
20 cmake --build books/cpu-volume-1-practice/build/executable-evidence-debug
21 ctest --test-dir books/cpu-volume-1-practice/build/executable-evidence-debug \
22       --output-on-failure
23 ```
24
25 ## 运行
26
27 ```bash
28 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
29   ↵ executable_evidence/cpu1ee_cli \
30 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
31   ↵ executable_evidence/cpu1ee_cli
32 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
33   ↵ executable_evidence/cpu1ee_cli \
34 books/cpu-volume-1-practice/build/executable-evidence-debug/labs/↵
35   ↵ executable_evidence/cpu1ee_cli \
36 --sections-csv
37 ```
38
39 ## 研究项目
40
41 最终项目不是“看懂 ELF 文件格式”这么窄，而是提交一份二进制证据报告：
42
43 - 选一个自己编译的小程序；
44 - 用 `cpu1ee_cli` 记录 header、sections、symbols 和 checksum；
45 - 用 `readelf` 或 `objdump` 交叉验证至少三个字段；
46 - 对一个 C++ 函数解释它的符号、section、入口地址或机器码位置；
47 - 说明 Debug/Release、优化等级或 strip 对报告有什么影响；
48 - 把不可验证的结论写成限制，不写成事实。

```

Listing 13.3 Executable Evidence Kit CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CPU1ExecutableEvidence LANGUAGES CXX)
3
4 list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_LIST_DIR}/../../../../cmake")
5 include(BookLabDependencies)
6 book_labs_enable_gtest()
7 book_labs_enable_benchmark()
8
9 add_library(cpu1_executable_evidence STATIC
10     src/executable_evidence.cpp
11 )
12
13 target_include_directories(cpu1_executable_evidence
14     PUBLIC
15     ${CMAKE_CURRENT_SOURCE_DIR}/include

```

```

16 )
17
18 target_compile_features(cpu1_executable_evidence PUBLIC cxx_std_20)
19
20 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
21     target_compile_options(cpu1_executable_evidence PRIVATE -Wall -Wextra -Wpedantic)
22 endif()
23
24 add_executable(cpu1ee_cli
25     src/main.cpp
26 )
27
28 target_link_libraries(cpu1ee_cli PRIVATE cpu1_executable_evidence)
29
30 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
31     target_compile_options(cpu1ee_cli PRIVATE -Wall -Wextra -Wpedantic)
32 endif()
33
34 add_executable(cpu1ee_tests
35     tests/executable_evidence_tests.cpp
36 )
37
38 target_link_libraries(cpu1ee_tests
39     PRIVATE
40         cpu1_executable_evidence
41         GTest::gmock_main
42 )
43
44 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
45     target_compile_options(cpu1ee_tests PRIVATE -Wall -Wextra -Wpedantic)
46 endif()
47
48 add_test(NAME cpu1ee_tests COMMAND ${CMAKE_TARGET_FILE:cpu1ee_tests})
49
50 add_executable(cpu1ee_bench
51     bench/executable_evidence_bench.cpp
52 )
53
54 target_link_libraries(cpu1ee_bench
55     PRIVATE
56         cpu1_executable_evidence
57         benchmark::benchmark
58 )
59
60 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
61     target_compile_options(cpu1ee_bench PRIVATE -Wall -Wextra -Wpedantic)
62 endif()

```

二、公共合同

Listing 13.4 include/cpu1/executable_evidence.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <filesystem>
5  #include <span>
6  #include <string>
7  #include <string_view>
8  #include <vector>
9
10 namespace cpu1 {
11
12 struct InspectionConfig {
13     bool include_symbols{true};
14     std::uint64_t max_symbols{128};
15 };
16
17 struct SectionSummary {
18     std::string name;
19     std::uint32_t type{};
20     std::uint64_t flags{};
21     std::uint64_t address{};
22     std::uint64_t offset{};
23     std::uint64_t size{};
24 };
25
26 struct SymbolSummary {
27     std::string name;
28     std::string section_name;
29     std::uint8_t type{};
30     std::uint8_t binding{};
31     std::uint16_t section_index{};
32     std::uint64_t value{};
33     std::uint64_t size{};
34 };
35
36 struct ElfReport {
37     std::string source_name;
38     bool valid_magic{};
39     bool is_elf64{};
40     bool is_little_endian{};
41     std::uint16_t object_type{};
42     std::uint16_t machine{};
43     std::uint64_t entry_address{};
44     std::uint64_t checksum{};
45     std::uint64_t byte_size{};
46     std::vector<SectionSummary> sections;
47     std::vector<SymbolSummary> symbols;
48     std::vector<std::string> diagnostics;
49 };
50
51 std::vector<std::uint8_t> read_binary_file(const std::filesystem::path& path);

```

```

52 std::uint64_t checksum_bytes(std::span<const std::uint8_t> bytes);
53 std::string classify_machine(std::uint16_t machine);
54 ElfReport inspect_elf(
55     std::span<const std::uint8_t> bytes,
56     std::string_view source_name,
57     const InspectionConfig& config = {});
58 std::string format_report(const ElfReport& report);
59 std::string format_sections_csv(const ElfReport& report);
60
61 } // namespace cpu1

```

Listing 13.5 include/cpu1/elf_fixture.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <cstdint>
5 #include <string_view>
6 #include <vector>
7
8 namespace cpu1 {
9
10 constexpr std::size_t FIXTURE_ELF_SIZE = 704;
11 constexpr std::size_t FIXTURE_ELF_IDENT_SIZE = 16;
12 constexpr std::size_t FIXTURE_ELF_HEADER_SIZE = 64;
13 constexpr std::size_t FIXTURE_ELF_SH_OFFSET = 384;
14 constexpr std::size_t FIXTURE_ELF_SH_ENTRY_SIZE = 64;
15 constexpr std::size_t FIXTURE_ELF_SH_COUNT = 5;
16 constexpr std::size_t FIXTURE_ELF_SHSTR_INDEX = 2;
17 constexpr std::size_t FIXTURE_ELF_TEXT_OFFSET = 96;
18 constexpr std::size_t FIXTURE_ELF_TEXT_SIZE = 4;
19 constexpr std::size_t FIXTURE_ELF_SHSTRTAB_OFFSET = 128;
20 constexpr std::size_t FIXTURE_ELF_STRTAB_OFFSET = 176;
21 constexpr std::size_t FIXTURE_ELF_SYMTAB_OFFSET = 208;
22 constexpr std::size_t FIXTURE_ELF_SYMTAB_ENTRY_SIZE = 24;
23 constexpr std::uint64_t FIXTURE_ELF_ENTRY = 0x401000;
24 constexpr std::uint16_t FIXTURE_ELF_EXEC_TYPE = 2;
25 constexpr std::uint16_t FIXTURE_ELF_X86_64 = 62;
26 constexpr std::uint32_t FIXTURE_ELF_VERSION = 1;
27 constexpr std::uint32_t FIXTURE_ELF_PROGBITS = 1;
28 constexpr std::uint32_t FIXTURE_ELF_SYMTAB = 2;
29 constexpr std::uint32_t FIXTURE_ELF_STRTAB = 3;
30 constexpr std::uint64_t FIXTURE_ELF_ALLOC_EXEC_FLAGS = 6;
31 constexpr std::uint8_t FIXTURE_ELF64_CLASS = 2;
32 constexpr std::uint8_t FIXTURE_ELF_LITTLE_ENDIAN = 1;
33 constexpr std::uint8_t FIXTURE_ELF_MAGIC_0 = 0x7F;
34 constexpr std::uint8_t FIXTURE_ELF_MAGIC_1 = 'E';
35 constexpr std::uint8_t FIXTURE_ELF_MAGIC_2 = 'L';
36 constexpr std::uint8_t FIXTURE_ELF_MAGIC_3 = 'F';
37 constexpr std::uint8_t FIXTURE_SYMBOL_GLOBAL_FUNC = 0x12;
38 constexpr std::uint8_t FIXTURE_SYMBOL_LOCAL_FUNC = 0x02;
39 constexpr std::uint8_t FIXTURE_TEXT_BYTE_0 = 0x55;

```

```

40 constexpr std::uint8_t FIXTURE_TEXT_BYTE_1 = 0x48;
41 constexpr std::uint8_t FIXTURE_TEXT_BYTE_2 = 0x89;
42 constexpr std::uint8_t FIXTURE_TEXT_BYTE_3 = 0xE5;
43 constexpr std::size_t FIXTURE_ELF_TYPE_OFFSET = 16;
44 constexpr std::size_t FIXTURE_ELF_MACHINE_OFFSET = 18;
45 constexpr std::size_t FIXTURE_ELF_VERSION_OFFSET = 20;
46 constexpr std::size_t FIXTURE_ELF_ENTRY_OFFSET = 24;
47 constexpr std::size_t FIXTURE_ELF_SECTION_HEADER_OFFSET = 40;
48 constexpr std::size_t FIXTURE_ELF_HEADER_SIZE_OFFSET = 52;
49 constexpr std::size_t FIXTURE_ELF_SECTION_ENTRY_SIZE_OFFSET = 58;
50 constexpr std::size_t FIXTURE_ELF_SECTION_COUNT_OFFSET = 60;
51 constexpr std::size_t FIXTURE_ELF_SECTION_NAME_INDEX_OFFSET = 62;
52 constexpr std::size_t FIXTURE_SECTION_NAME_OFFSET = 0;
53 constexpr std::size_t FIXTURE_SECTION_TYPE_OFFSET = 4;
54 constexpr std::size_t FIXTURE_SECTION_FLAGS_OFFSET = 8;
55 constexpr std::size_t FIXTURE_SECTION_ADDRESS_OFFSET = 16;
56 constexpr std::size_t FIXTURE_SECTION_FILE_OFFSET = 24;
57 constexpr std::size_t FIXTURE_SECTION_SIZE_OFFSET = 32;
58 constexpr std::size_t FIXTURE_SECTION_LINK_OFFSET = 40;
59 constexpr std::size_t FIXTURE_SECTION_ENTRY_SIZE_OFFSET = 56;
60 constexpr std::size_t FIXTURE_SYMBOL_NAME_OFFSET = 0;
61 constexpr std::size_t FIXTURE_SYMBOL_INFO_OFFSET = 4;
62 constexpr std::size_t FIXTURE_SYMBOL_SECTION_INDEX_OFFSET = 6;
63 constexpr std::size_t FIXTURE_SYMBOL_VALUE_OFFSET = 8;
64 constexpr std::size_t FIXTURE_SYMBOL_SIZE_OFFSET = 16;
65
66 inline void put_u16(std::vector<std::uint8_t>& bytes, std::size_t offset, std::uint16_t value)
67 {
68     bytes[offset] = static_cast<std::uint8_t>(value & 0xFFU);
69     bytes[offset + 1] = static_cast<std::uint8_t>((value >> 8U) & 0xFFU);
70 }
71
72 inline void put_u32(std::vector<std::uint8_t>& bytes, std::size_t offset, std::uint32_t value)
73 {
74     constexpr std::size_t BYTE_BITS = 8;
75     for (std::size_t i = 0; i < sizeof(std::uint32_t); ++i) {
76         bytes[offset + i] = static_cast<std::uint8_t>((value >> (i * BYTE_BITS) & 0xFFU);
77     }
78 }
79
80 inline void put_u64(std::vector<std::uint8_t>& bytes, std::size_t offset, std::uint64_t value)
81 {
82     constexpr std::size_t BYTE_BITS = 8;
83     for (std::size_t i = 0; i < sizeof(std::uint64_t); ++i) {
84         bytes[offset + i] = static_cast<std::uint8_t>((value >> (i * BYTE_BITS) & 0xFFU);
85     }
86 }

```



```

87
88 inline void put_string(std::vector<std::uint8_t>& bytes, std::size_t offset, ↵
    ↵ std::string_view value)
89 {
90     for (std::size_t i = 0; i < value.size(); ++i) {
91         bytes[offset + i] = static_cast<std::uint8_t>(value[i]);
92     }
93 }
94
95 inline void put_section_header(
96     std::vector<std::uint8_t>& bytes,
97     std::size_t index,
98     std::uint32_t name_offset,
99     std::uint32_t type,
100     std::uint64_t flags,
101     std::uint64_t address,
102     std::uint64_t file_offset,
103     std::uint64_t size,
104     std::uint32_t link,
105     std::uint64_t entry_size)
106 {
107     const std::size_t offset = FIXTURE_ELF_SH_OFFSET + index * ↵
    ↵ FIXTURE_ELF_SH_ENTRY_SIZE;
108     put_u32(bytes, offset + FIXTURE_SECTION_NAME_OFFSET, name_offset);
109     put_u32(bytes, offset + FIXTURE_SECTION_TYPE_OFFSET, type);
110     put_u64(bytes, offset + FIXTURE_SECTION_FLAGS_OFFSET, flags);
111     put_u64(bytes, offset + FIXTURE_SECTION_ADDRESS_OFFSET, address);
112     put_u64(bytes, offset + FIXTURE_SECTION_FILE_OFFSET, file_offset);
113     put_u64(bytes, offset + FIXTURE_SECTION_SIZE_OFFSET, size);
114     put_u32(bytes, offset + FIXTURE_SECTION_LINK_OFFSET, link);
115     put_u64(bytes, offset + FIXTURE_SECTION_ENTRY_SIZE_OFFSET, entry_size);
116 }
117
118 inline void put_symbol(
119     std::vector<std::uint8_t>& bytes,
120     std::size_t index,
121     std::uint32_t name_offset,
122     std::uint8_t info,
123     std::uint16_t section_index,
124     std::uint64_t value,
125     std::uint64_t size)
126 {
127     const std::size_t offset = FIXTURE_ELF_SYMTAB_OFFSET + index * ↵
    ↵ FIXTURE_ELF_SYMTAB_ENTRY_SIZE;
128     put_u32(bytes, offset + FIXTURE_SYMBOL_NAME_OFFSET, name_offset);
129     bytes[offset + FIXTURE_SYMBOL_INFO_OFFSET] = info;
130     put_u16(bytes, offset + FIXTURE_SYMBOL_SECTION_INDEX_OFFSET, section_index)↵
    ↵ ;
131     put_u64(bytes, offset + FIXTURE_SYMBOL_VALUE_OFFSET, value);
132     put_u64(bytes, offset + FIXTURE_SYMBOL_SIZE_OFFSET, size);
133 }
134

```

```

135 inline std::vector<std::uint8_t> make_synthetic_elf()
136 {
137     std::vector<std::uint8_t> bytes(FIXTURE_ELF_SIZE, 0);
138     bytes[0] = FIXTURE_ELF_MAGIC_0;
139     bytes[1] = FIXTURE_ELF_MAGIC_1;
140     bytes[2] = FIXTURE_ELF_MAGIC_2;
141     bytes[3] = FIXTURE_ELF_MAGIC_3;
142     bytes[4] = FIXTURE_ELF64_CLASS;
143     bytes[5] = FIXTURE_ELF_LITTLE_ENDIAN;
144     bytes[6] = 1;
145
146     put_u16(bytes, FIXTURE_ELF_TYPE_OFFSET, FIXTURE_ELF_EXEC_TYPE);
147     put_u16(bytes, FIXTURE_ELF_MACHINE_OFFSET, FIXTURE_ELF_X86_64);
148     put_u32(bytes, FIXTURE_ELF_VERSION_OFFSET, FIXTURE_ELF_VERSION);
149     put_u64(bytes, FIXTURE_ELF_ENTRY_OFFSET, FIXTURE_ELF_ENTRY);
150     put_u64(bytes, FIXTURE_ELF_SECTION_HEADER_OFFSET, FIXTURE_ELF_SH_OFFSET);
151     put_u16(bytes, FIXTURE_ELF_HEADER_SIZE_OFFSET, FIXTURE_ELF_HEADER_SIZE);
152     put_u16(bytes, FIXTURE_ELF_SECTION_ENTRY_SIZE_OFFSET, ←
    ↪ FIXTURE_ELF_SH_ENTRY_SIZE);
153     put_u16(bytes, FIXTURE_ELF_SECTION_COUNT_OFFSET, FIXTURE_ELF_SH_COUNT);
154     put_u16(bytes, FIXTURE_ELF_SECTION_NAME_INDEX_OFFSET, ←
    ↪ FIXTURE_ELF_SHSTR_INDEX);
155
156     bytes[FIXTURE_ELF_TEXT_OFFSET + 0] = FIXTURE_TEXT_BYTE_0;
157     bytes[FIXTURE_ELF_TEXT_OFFSET + 1] = FIXTURE_TEXT_BYTE_1;
158     bytes[FIXTURE_ELF_TEXT_OFFSET + 2] = FIXTURE_TEXT_BYTE_2;
159     bytes[FIXTURE_ELF_TEXT_OFFSET + 3] = FIXTURE_TEXT_BYTE_3;
160
161     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 1, ".text");
162     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 7, ".shstrtab");
163     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 17, ".symtab");
164     put_string(bytes, FIXTURE_ELF_SHSTRTAB_OFFSET + 25, ".strtab");
165     put_string(bytes, FIXTURE_ELF_STRTAB_OFFSET + 1, "_start");
166     put_string(bytes, FIXTURE_ELF_STRTAB_OFFSET + 8, "helper");
167
168     put_symbol(bytes, 1, 1, FIXTURE_SYMBOL_GLOBAL_FUNC, 1, FIXTURE_ELF_ENTRY,
169                FIXTURE_ELF_TEXT_SIZE);
170     put_symbol(bytes, 2, 8, FIXTURE_SYMBOL_LOCAL_FUNC, 1,
171                FIXTURE_ELF_ENTRY + FIXTURE_ELF_TEXT_SIZE, 8);
172
173     put_section_header(bytes, 1, 1, FIXTURE_ELF_PROGBITS, ←
    ↪ FIXTURE_ELF_ALLOC_EXEC_FLAGS,
174                FIXTURE_ELF_ENTRY, FIXTURE_ELF_TEXT_OFFSET, ←
    ↪ FIXTURE_ELF_TEXT_SIZE, 0, 0);
175     put_section_header(bytes, 2, 7, FIXTURE_ELF_STRTAB, 0, 0, ←
    ↪ FIXTURE_ELF_SHSTRTAB_OFFSET, 33, 0,
176                0);
177     put_section_header(bytes, 3, 17, FIXTURE_ELF_SYMTAB, 0, 0, ←
    ↪ FIXTURE_ELF_SYMTAB_OFFSET,
178                FIXTURE_ELF_SYMTAB_ENTRY_SIZE * 3, 4, ←
    ↪ FIXTURE_ELF_SYMTAB_ENTRY_SIZE);
179     put_section_header(bytes, 4, 25, FIXTURE_ELF_STRTAB, 0, 0, ←

```

```

    ↪ FIXTURE_ELF_STRTAB_OFFSET, 15, 0,
180         0);
181
182     return bytes;
183 }
184
185 } // namespace cpu1

```

三、实现和 CLI

Listing 13.6 src/executable_evidence.cpp

```

1  #include <cpu1/executable_evidence.hpp>
2
3  #include <algorithm>
4  #include <cstdint>
5  #include <fstream>
6  #include <iomanip>
7  #include <sstream>
8  #include <stdexcept>
9
10 namespace cpu1 {
11 namespace {
12
13 constexpr std::uint8_t ELF_MAGIC_0 = 0x7F;
14 constexpr std::uint8_t ELF_MAGIC_1 = 'E';
15 constexpr std::uint8_t ELF_MAGIC_2 = 'L';
16 constexpr std::uint8_t ELF_MAGIC_3 = 'F';
17 constexpr std::size_t ELF_IDENT_SIZE = 16;
18 constexpr std::size_t ELF_IDENT_CLASS_OFFSET = 4;
19 constexpr std::size_t ELF_IDENT_DATA_OFFSET = 5;
20 constexpr std::uint8_t ELF_CLASS_64 = 2;
21 constexpr std::uint8_t ELF_DATA_LITTLE_ENDIAN = 1;
22
23 constexpr std::size_t ELF64_HEADER_MIN_SIZE = 64;
24 constexpr std::size_t ELF64_TYPE_OFFSET = 16;
25 constexpr std::size_t ELF64_MACHINE_OFFSET = 18;
26 constexpr std::size_t ELF64_ENTRY_OFFSET = 24;
27 constexpr std::size_t ELF64_SECTION_HEADER_OFFSET = 40;
28 constexpr std::size_t ELF64_HEADER_SECTION_ENTRY_SIZE_OFFSET = 58;
29 constexpr std::size_t ELF64_SECTION_COUNT_OFFSET = 60;
30 constexpr std::size_t ELF64_SECTION_NAME_INDEX_OFFSET = 62;
31
32 constexpr std::size_t ELF64_SECTION_NAME_OFFSET = 0;
33 constexpr std::size_t ELF64_SECTION_TYPE_OFFSET = 4;
34 constexpr std::size_t ELF64_SECTION_FLAGS_OFFSET = 8;
35 constexpr std::size_t ELF64_SECTION_ADDRESS_OFFSET = 16;
36 constexpr std::size_t ELF64_SECTION_FILE_OFFSET = 24;
37 constexpr std::size_t ELF64_SECTION_SIZE_OFFSET = 32;
38 constexpr std::size_t ELF64_SECTION_LINK_OFFSET = 40;
39 constexpr std::size_t ELF64_SECTION_ENTRY_SIZE_OFFSET = 56;

```

```

40 constexpr std::size_t ELF64_SECTION_HEADER_SIZE = 64;
41
42 constexpr std::size_t ELF64_SYMBOL_NAME_OFFSET = 0;
43 constexpr std::size_t ELF64_SYMBOL_INFO_OFFSET = 4;
44 constexpr std::size_t ELF64_SYMBOL_SECTION_INDEX_OFFSET = 6;
45 constexpr std::size_t ELF64_SYMBOL_VALUE_OFFSET = 8;
46 constexpr std::size_t ELF64_SYMBOL_SIZE_OFFSET = 16;
47 constexpr std::size_t ELF64_SYMBOL_ENTRY_SIZE = 24;
48
49 constexpr std::uint32_t ELF_SECTION_TYPE_SYMTAB = 2;
50 constexpr std::uint32_t ELF_SECTION_TYPE_DYNSYM = 11;
51 constexpr std::uint16_t ELF_MACHINE_X86_64 = 62;
52 constexpr std::uint16_t ELF_MACHINE_AARCH64 = 183;
53 constexpr std::uint16_t ELF_MACHINE_RISCV = 243;
54
55 constexpr std::uint64_t FNV_OFFSET_BASIS = 14695981039346656037ULL;
56 constexpr std::uint64_t FNV_PRIME = 1099511628211ULL;
57 constexpr std::uint64_t HEX_FIELD_WIDTH_64 = 16;
58
59 bool has_range(std::span<const std::uint8_t> bytes, std::uint64_t offset, std::uint64_t size)
60 {
61     const std::uint64_t byte_count = static_cast<std::uint64_t>(bytes.size());
62     return offset <= byte_count && size <= byte_count - offset;
63 }
64
65 std::uint8_t read_u8(std::span<const std::uint8_t> bytes, std::uint64_t offset)
66 {
67     if (!has_range(bytes, offset, sizeof(std::uint8_t))) {
68         throw std::out_of_range("read_u8 outside input");
69     }
70     return bytes[static_cast<std::size_t>(offset)];
71 }
72
73 std::uint16_t read_u16_le(std::span<const std::uint8_t> bytes, std::uint64_t offset)
74 {
75     if (!has_range(bytes, offset, sizeof(std::uint16_t))) {
76         throw std::out_of_range("read_u16_le outside input");
77     }
78     const std::uint16_t low = bytes[static_cast<std::size_t>(offset)];
79     const std::uint16_t high = bytes[static_cast<std::size_t>(offset + 1)];
80     return static_cast<std::uint16_t>(low | static_cast<std::uint16_t>(high << 8U));
81 }
82
83 std::uint32_t read_u32_le(std::span<const std::uint8_t> bytes, std::uint64_t offset)
84 {
85     if (!has_range(bytes, offset, sizeof(std::uint32_t))) {
86         throw std::out_of_range("read_u32_le outside input");
87     }

```

```

88     std::uint32_t value = 0;
89     constexpr std::uint32_t BYTE_BITS = 8;
90     for (std::uint32_t i = 0; i < sizeof(std::uint32_t); ++i) {
91         value |= static_cast<std::uint32_t>(bytes[static_cast<std::size_t>(↵
↵ offset + i)])
92             << (i * BYTE_BITS);
93     }
94     return value;
95 }
96
97 std::uint64_t read_u64_le(std::span<const std::uint8_t> bytes, std::uint64_t ↵
↵ offset)
98 {
99     if (!has_range(bytes, offset, sizeof(std::uint64_t))) {
100         throw std::out_of_range("read_u64_le outside input");
101     }
102     std::uint64_t value = 0;
103     constexpr std::uint64_t BYTE_BITS = 8;
104     for (std::uint64_t i = 0; i < sizeof(std::uint64_t); ++i) {
105         value |= static_cast<std::uint64_t>(bytes[static_cast<std::size_t>(↵
↵ offset + i)])
106             << (i * BYTE_BITS);
107     }
108     return value;
109 }
110
111 void write_hex64(std::ostream& output, std::uint64_t value)
112 {
113     output << "0x" << std::hex << std::setfill('0') << std::setw(↵
↵ HEX_FIELD_WIDTH_64) << value
114         << std::dec << std::setfill(' ');
115 }
116
117 std::string read_string(std::span<const std::uint8_t> table, std::uint32_t ↵
↵ offset)
118 {
119     if (offset >= table.size()) {
120         return {};
121     }
122
123     std::string value;
124     for (std::size_t i = offset; i < table.size() && table[i] != 0; ++i) {
125         value.push_back(static_cast<char>(table[i]));
126     }
127     return value;
128 }
129
130 std::span<const std::uint8_t> slice_bytes(
131     std::span<const std::uint8_t> bytes,
132     std::uint64_t offset,
133     std::uint64_t size)
134 {

```

```

135     if (!has_range(bytes, offset, size)) {
136         return {};
137     }
138     return bytes.subspan(static_cast<std::size_t>(offset), static_cast<std::size_t>(size));
139 }
140
141 struct RawSection {
142     std::uint32_t name_offset{};
143     std::uint32_t type{};
144     std::uint64_t flags{};
145     std::uint64_t address{};
146     std::uint64_t offset{};
147     std::uint64_t size{};
148     std::uint32_t link{};
149     std::uint64_t entry_size{};
150 };
151
152 RawSection read_raw_section(std::span<const std::uint8_t> bytes, std::uint64_t section_offset)
153 {
154     return RawSection{
155         .name_offset = read_u32_le(bytes, section_offset + ELF64_SECTION_NAME_OFFSET),
156         .type = read_u32_le(bytes, section_offset + ELF64_SECTION_TYPE_OFFSET),
157         .flags = read_u64_le(bytes, section_offset + ELF64_SECTION_FLAGS_OFFSET),
158         .address = read_u64_le(bytes, section_offset + ELF64_SECTION_ADDRESS_OFFSET),
159         .offset = read_u64_le(bytes, section_offset + ELF64_SECTION_FILE_OFFSET),
160         .size = read_u64_le(bytes, section_offset + ELF64_SECTION_SIZE_OFFSET),
161         .link = read_u32_le(bytes, section_offset + ELF64_SECTION_LINK_OFFSET),
162         .entry_size = read_u64_le(bytes, section_offset + ELF64_SECTION_ENTRY_SIZE_OFFSET),
163     };
164 }
165
166 std::vector<RawSection> read_raw_sections(
167     std::span<const std::uint8_t> bytes,
168     std::uint64_t section_header_offset,
169     std::uint16_t section_count,
170     std::uint16_t section_entry_size,
171     std::vector<std::string>& diagnostics)
172 {
173     std::vector<RawSection> sections;
174     if (section_entry_size < ELF64_SECTION_HEADER_SIZE) {
175         diagnostics.push_back("section entry size is smaller than ELF64 section header");
176         return sections;
177     }
178 }

```

```

179     sections.reserve(section_count);
180     for (std::uint16_t index = 0; index < section_count; ++index) {
181         const std::uint64_t current_offset =
182             section_header_offset + static_cast<std::uint64_t>(index) * ↵
↵ section_entry_size;
183         if (!has_range(bytes, current_offset, ELF64_SECTION_HEADER_SIZE)) {
184             diagnostics.push_back("section header table extends beyond input");
185             break;
186         }
187         sections.push_back(read_raw_section(bytes, current_offset));
188     }
189     return sections;
190 }
191
192 std::vector<SectionSummary> summarize_sections(
193     std::span<const std::uint8_t> bytes,
194     const std::vector<RawSection>& raw_sections,
195     std::uint16_t section_name_index,
196     std::vector<std::string>& diagnostics)
197 {
198     if (section_name_index >= raw_sections.size()) {
199         diagnostics.push_back("section-name string table index is out of range"↵
↵ );
200     }
201     return {};
202 }
203
204 const RawSection& name_section = raw_sections[section_name_index];
205 const std::span<const std::uint8_t> names =
206     slice_bytes(bytes, name_section.offset, name_section.size);
207 if (names.empty() && name_section.size != 0) {
208     diagnostics.push_back("section-name string table extends beyond input"↵
↵ );
209     return {};
210 }
211
212 std::vector<SectionSummary> summaries;
213 summaries.reserve(raw_sections.size());
214 for (const RawSection& raw : raw_sections) {
215     summaries.push_back(SectionSummary{
216         .name = read_string(names, raw.name_offset),
217         .type = raw.type,
218         .flags = raw.flags,
219         .address = raw.address,
220         .offset = raw.offset,
221         .size = raw.size,
222     });
223 }
224 return summaries;
225 }
226
227 std::vector<SymbolSummary> summarize_symbols(
228     std::span<const std::uint8_t> bytes,

```

```

228     const std::vector<RawSection>& raw_sections,
229     const std::vector<SectionSummary>& sections,
230     const InspectionConfig& config,
231     std::vector<std::string>& diagnostics)
232 {
233     std::vector<SymbolSummary> symbols;
234     if (!config.include_symbols) {
235         return symbols;
236     }
237
238     for (std::size_t section_index = 0; section_index < raw_sections.size(); ++↵
↵ section_index) {
239         const RawSection& symbol_section = raw_sections[section_index];
240         if (symbol_section.type != ELF_SECTION_TYPE_SYMTAB &&
241             symbol_section.type != ELF_SECTION_TYPE_DYNSYM) {
242             continue;
243         }
244         if (symbol_section.entry_size < ELF64_SYMBOL_ENTRY_SIZE) {
245             diagnostics.push_back("symbol entry size is smaller than ELF64 ↵
↵ symbol entry");
246             continue;
247         }
248         if (symbol_section.link >= raw_sections.size()) {
249             diagnostics.push_back("symbol string table link is out of range");
250             continue;
251         }
252
253         const RawSection& string_section = raw_sections[symbol_section.link];
254         const std::span<const std::uint8_t> symbol_names =
255             slice_bytes(bytes, string_section.offset, string_section.size);
256         const std::uint64_t symbol_count = symbol_section.size / symbol_section↵
↵ .entry_size;
257         const std::uint64_t capped_count = std::min(symbol_count, config.↵
↵ max_symbols);
258
259         for (std::uint64_t symbol_index = 0; symbol_index < capped_count; ++↵
↵ symbol_index) {
260             const std::uint64_t symbol_offset =
261                 symbol_section.offset + symbol_index * symbol_section.↵
↵ entry_size;
262             if (!has_range(bytes, symbol_offset, ELF64_SYMBOL_ENTRY_SIZE)) {
263                 diagnostics.push_back("symbol table extends beyond input");
264                 break;
265             }
266
267             const std::uint32_t name_offset =
268                 read_u32_le(bytes, symbol_offset + ELF64_SYMBOL_NAME_OFFSET);
269             const std::uint8_t info = read_u8(bytes, symbol_offset + ↵
↵ ELF64_SYMBOL_INFO_OFFSET);
270             const std::uint16_t symbol_section_index =
271                 read_u16_le(bytes, symbol_offset + ↵
↵ ELF64_SYMBOL_SECTION_INDEX_OFFSET);

```



```

272         const std::string section_name =
273             symbol_section_index < sections.size() ? sections[
↳ symbol_section_index].name : "";
274
275         symbols.push_back(SymbolSummary{
276             .name = read_string(symbol_names, name_offset),
277             .section_name = section_name,
278             .type = static_cast<std::uint8_t>(info & 0x0FU),
279             .binding = static_cast<std::uint8_t>(info >> 4U),
280             .section_index = symbol_section_index,
281             .value = read_u64_le(bytes, symbol_offset +
↳ ELF64_SYMBOL_VALUE_OFFSET),
282             .size = read_u64_le(bytes, symbol_offset +
↳ ELF64_SYMBOL_SIZE_OFFSET),
283         });
284     }
285
286     if (symbol_count > config.max_symbols) {
287         diagnostics.push_back("symbol table truncated by max_symbols");
288     }
289 }
290
291 return symbols;
292 }
293
294 bool has_elf_magic(std::span<const std::uint8_t> bytes)
295 {
296     return bytes.size() >= ELF_IDENT_SIZE && bytes[0] == ELF_MAGIC_0 && bytes
↳ [1] == ELF_MAGIC_1 &&
↳ bytes[2] == ELF_MAGIC_2 && bytes[3] == ELF_MAGIC_3;
297 }
298
299
300 } // namespace
301
302 std::vector<std::uint8_t> read_binary_file(const std::filesystem::path& path)
303 {
304     std::ifstream input(path, std::ios::binary);
305     if (!input) {
306         throw std::runtime_error("failed to open binary file: " + path.string()
↳ );
307     }
308
309     input.seekg(0, std::ios::end);
310     const std::streamoff size = input.tellg();
311     if (size < 0) {
312         throw std::runtime_error("failed to determine binary file size: " +
↳ path.string());
313     }
314     input.seekg(0, std::ios::beg);
315
316     std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size));
317     if (!bytes.empty()) {

```

```

318     input.read(reinterpret_cast<char*>(bytes.data()), size);
319 }
320 if (!input && !input.eof()) {
321     throw std::runtime_error("failed to read binary file: " + path.string()↵
↵ );
322 }
323 return bytes;
324 }
325
326 std::uint64_t checksum_bytes(std::span<const std::uint8_t> bytes)
327 {
328     std::uint64_t value = FNV_OFFSET_BASIS;
329     for (const std::uint8_t byte : bytes) {
330         value ^= byte;
331         value *= FNV_PRIME;
332     }
333     return value;
334 }
335
336 std::string classify_machine(std::uint16_t machine)
337 {
338     switch (machine) {
339     case ELF_MACHINE_X86_64:
340         return "x86-64";
341     case ELF_MACHINE_AARCH64:
342         return "AArch64";
343     case ELF_MACHINE_RISCV:
344         return "RISC-V";
345     default:
346         return "unknown";
347     }
348 }
349
350 ElfReport inspect_elf(
351     std::span<const std::uint8_t> bytes,
352     std::string_view source_name,
353     const InspectionConfig& config)
354 {
355     ElfReport report;
356     report.source_name = std::string(source_name);
357     report.byte_size = bytes.size();
358     report.checksum = checksum_bytes(bytes);
359     report.valid_magic = has_elf_magic(bytes);
360
361     if (!report.valid_magic) {
362         report.diagnostics.push_back("input does not start with ELF magic");
363         return report;
364     }
365     if (bytes.size() < ELF64_HEADER_MIN_SIZE) {
366         report.diagnostics.push_back("input is shorter than ELF64 header");
367         return report;
368     }

```

```

369
370     report.is_elf64 = read_u8(bytes, ELF_IDENT_CLASS_OFFSET) == ELF_CLASS_64;
371     report.is_little_endian = read_u8(bytes, ELF_IDENT_DATA_OFFSET) == ↵
↵ ELF_DATA_LITTLE_ENDIAN;
372     if (!report.is_elf64 || !report.is_little_endian) {
373         report.diagnostics.push_back("only ELF64 little-endian inputs are ↵
↵ supported");
374         return report;
375     }
376
377     report.object_type = read_u16_le(bytes, ELF64_TYPE_OFFSET);
378     report.machine = read_u16_le(bytes, ELF64_MACHINE_OFFSET);
379     report.entry_address = read_u64_le(bytes, ELF64_ENTRY_OFFSET);
380
381     const std::uint64_t section_header_offset = read_u64_le(bytes, ↵
↵ ELF64_SECTION_HEADER_OFFSET);
382     const std::uint16_t section_entry_size =
383         read_u16_le(bytes, ELF64_HEADER_SECTION_ENTRY_SIZE_OFFSET);
384     const std::uint16_t section_count = read_u16_le(bytes, ↵
↵ ELF64_SECTION_COUNT_OFFSET);
385     const std::uint16_t section_name_index = read_u16_le(bytes, ↵
↵ ELF64_SECTION_NAME_INDEX_OFFSET);
386
387     std::vector<RawSection> raw_sections = read_raw_sections(
388         bytes,
389         section_header_offset,
390         section_count,
391         section_entry_size,
392         report.diagnostics);
393     report.sections = summarize_sections(bytes, raw_sections, ↵
↵ section_name_index, report.diagnostics);
394     report.symbols =
395         summarize_symbols(bytes, raw_sections, report.sections, config, report.↵
↵ diagnostics);
396
397     return report;
398 }
399
400 std::string format_report(const ElfReport& report)
401 {
402     std::ostringstream output;
403     output << "source=" << report.source_name << '\n';
404     output << "bytes=" << report.byte_size << '\n';
405     output << "checksum=" << report.checksum << '\n';
406     output << "valid_magic=" << (report.valid_magic ? "true" : "false") << '\n'↵
↵ ;
407     output << "class=" << (report.is_elf64 ? "ELF64" : "unsupported") << '\n';
408     output << "endian=" << (report.is_little_endian ? "little" : "unsupported")↵
↵ << '\n';
409     output << "machine=" << classify_machine(report.machine) << '\n';
410     output << "entry=";
411     write_hex64(output, report.entry_address);

```

```

412     output << '\n';
413     output << "sections=" << report.sections.size() << '\n';
414     output << "symbols=" << report.symbols.size() << '\n';
415     for (const std::string& diagnostic : report.diagnostics) {
416         output << "diagnostic=" << diagnostic << '\n';
417     }
418     return output.str();
419 }
420
421 std::string format_sections_csv(const ElfReport& report)
422 {
423     std::ostringstream output;
424     output << "name,type,flags,address,offset,size\n";
425     for (const SectionSummary& section : report.sections) {
426         output << section.name << ',' << section.type << ',' << section.flags <
↵ << ','
427         << section.address << ',' << section.offset << ',' << section.↵
↵ size << '\n';
428     }
429     return output.str();
430 }
431
432 } // namespace cpu1

```

Listing 13.7 src/main.cpp

```

1  #include <cpu1/executable_evidence.hpp>
2
3  #include <exception>
4  #include <iostream>
5  #include <string_view>
6
7  namespace {
8
9  constexpr int CLI_SUCCESS = 0;
10 constexpr int CLI_USAGE_ERROR = 2;
11 constexpr int CLI_RUNTIME_ERROR = 1;
12 constexpr int CLI_MIN_ARGUMENTS = 2;
13 constexpr int CLI_MODE_ARGUMENTS = 3;
14
15 void print_usage(std::ostream& output)
16 {
17     output << "usage: cpu1ee_cli <elf-file> [--sections-csv]\n";
18 }
19
20 } // namespace
21
22 int main(int argc, char** argv)
23 {
24     if (argc < CLI_MIN_ARGUMENTS || argc > CLI_MODE_ARGUMENTS) {
25         print_usage(std::cerr);
26         return CLI_USAGE_ERROR;

```

```

27     }
28
29     try {
30         const std::vector<std::uint8_t> bytes = cpu1::read_binary_file(argv[1])↵
↵ ;
31         const cpu1::ElfReport report = cpu1::inspect_elf(bytes, argv[1]);
32
33         if (argc == CLI_MODE_ARGUMENTS && std::string_view(argv[2]) == "--↵
↵ sections-csv") {
34             std::cout << cpu1::format_sections_csv(report);
35             return CLI_SUCCESS;
36         }
37
38         std::cout << cpu1::format_report(report);
39         return CLI_SUCCESS;
40     } catch (const std::exception& error) {
41         std::cerr << "cpu1ee_cli: " << error.what() << '\n';
42         return CLI_RUNTIME_ERROR;
43     }
44 }

```

四、测试和 Benchmark

Listing 13.8 tests/executable_evidence_tests.cpp

```

1 #include <cpu1/elf_fixture.hpp>
2 #include <cpu1/executable_evidence.hpp>
3
4 #include <gmock/gmock.h>
5 #include <gtest/gtest.h>
6
7 #include <cstdint>
8 #include <filesystem>
9 #include <fstream>
10 #include <string>
11 #include <vector>
12
13 namespace {
14
15 using ::testing::HasSubstr;
16
17 constexpr std::uint16_t TEST_ELF_MACHINE_AARCH64 = 183;
18 constexpr std::uint16_t TEST_ELF_MACHINE_RISCV = 243;
19 constexpr std::uint16_t TEST_ELF_MACHINE_UNKNOWN = 0;
20
21 } // namespace
22
23 TEST(CPU1ExecutableEvidenceTest, ParsesSyntheticElfHeaderSectionsAndSymbols)
24 {
25     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
26     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "fixture.elf");

```

```

27
28 EXPECT_TRUE(report.valid_magic);
29 EXPECT_TRUE(report.is_elf64);
30 EXPECT_TRUE(report.is_little_endian);
31 EXPECT_EQ(report.object_type, cpu1::FIXTURE_ELF_EXEC_TYPE);
32 EXPECT_EQ(report.machine, cpu1::FIXTURE_ELF_X86_64);
33 EXPECT_EQ(report.entry_address, cpu1::FIXTURE_ELF_ENTRY);
34 EXPECT_EQ(report.byte_size, cpu1::FIXTURE_ELF_SIZE);
35 EXPECT_TRUE(report.diagnostics.empty());
36 ASSERT_EQ(report.sections.size(), cpu1::FIXTURE_ELF_SH_COUNT);
37 EXPECT_EQ(report.sections[1].name, ".text");
38 EXPECT_EQ(report.sections[1].offset, cpu1::FIXTURE_ELF_TEXT_OFFSET);
39 EXPECT_EQ(report.sections[1].size, cpu1::FIXTURE_ELF_TEXT_SIZE);
40 ASSERT_EQ(report.symbols.size(), 3U);
41 EXPECT_EQ(report.symbols[1].name, "_start");
42 EXPECT_EQ(report.symbols[1].section_name, ".text");
43 EXPECT_EQ(report.symbols[1].binding, 1U);
44 EXPECT_EQ(report.symbols[1].type, 2U);
45 }
46
47 TEST(CPU1ExecutableEvidenceTest, FormatsStableReportAndSectionCsv)
48 {
49     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
50     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "fixture.elf");
51
52     const std::string text = cpu1::format_report(report);
53     const std::string csv = cpu1::format_sections_csv(report);
54
55     EXPECT_THAT(text, HasSubstr("source=fixture.elf"));
56     EXPECT_THAT(text, HasSubstr("machine=x86-64"));
57     EXPECT_THAT(text, HasSubstr("sections=5"));
58     EXPECT_THAT(text, HasSubstr("symbols=3"));
59     EXPECT_THAT(csv, HasSubstr("name,type,flags,address,offset,size"));
60     EXPECT_THAT(csv, HasSubstr(".text,1,6,4198400,96,4"));
61 }
62
63 TEST(CPU1ExecutableEvidenceTest, RejectsNonElfMagic)
64 {
65     const std::vector<std::uint8_t> bytes{0, 1, 2, 3};
66     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "bad.bin");
67
68     EXPECT_FALSE(report.valid_magic);
69     ASSERT_EQ(report.diagnostics.size(), 1U);
70     EXPECT_EQ(report.diagnostics[0], "input does not start with ELF magic");
71 }
72
73 TEST(CPU1ExecutableEvidenceTest, RejectsShortElfHeader)
74 {
75     std::vector<std::uint8_t> bytes(cpu1::FIXTURE_ELF_IDENT_SIZE, 0);
76     bytes[0] = cpu1::FIXTURE_ELF_MAGIC_0;
77     bytes[1] = cpu1::FIXTURE_ELF_MAGIC_1;
78     bytes[2] = cpu1::FIXTURE_ELF_MAGIC_2;

```

```

79     bytes[3] = cpu1::FIXTURE_ELF_MAGIC_3;
80
81     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "short.elf");
82
83     EXPECT_TRUE(report.valid_magic);
84     ASSERT_EQ(report.diagnostics.size(), 1U);
85     EXPECT_EQ(report.diagnostics[0], "input is shorter than ELF64 header");
86 }
87
88 TEST(CPU1ExecutableEvidenceTest, RejectsUnsupportedClassOrEndian)
89 {
90     std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
91     bytes[4] = 1;
92
93     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "elf32.elf");
94
95     EXPECT_TRUE(report.valid_magic);
96     EXPECT_FALSE(report.is_elf64);
97     ASSERT_EQ(report.diagnostics.size(), 1U);
98     EXPECT_EQ(report.diagnostics[0], "only ELF64 little-endian inputs are ←
    ↪ supported");
99 }
100
101 TEST(CPU1ExecutableEvidenceTest, RespectsSymbolCap)
102 {
103     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
104     cpu1::InspectionConfig config;
105     config.max_symbols = 1;
106
107     const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "fixture.elf", ←
    ↪ config);
108
109     EXPECT_EQ(report.symbols.size(), 1U);
110     ASSERT_EQ(report.diagnostics.size(), 1U);
111     EXPECT_EQ(report.diagnostics[0], "symbol table truncated by max_symbols");
112 }
113
114 TEST(CPU1ExecutableEvidenceTest, ClassifiesKnownAndUnknownMachines)
115 {
116     EXPECT_EQ(cpu1::classify_machine(cpu1::FIXTURE_ELF_X86_64), "x86-64");
117     EXPECT_EQ(cpu1::classify_machine(TEST_ELF_MACHINE_AARCH64), "AArch64");
118     EXPECT_EQ(cpu1::classify_machine(TEST_ELF_MACHINE_RISCV), "RISC-V");
119     EXPECT_EQ(cpu1::classify_machine(TEST_ELF_MACHINE_UNKNOWN), "unknown");
120 }
121
122 TEST(CPU1ExecutableEvidenceTest, ReadsBinaryFileAndKeepsChecksumStable)
123 {
124     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
125     const std::filesystem::path path =
126         std::filesystem::temp_directory_path() / "cpu1ee_fixture.elf";
127     {
128         std::ofstream output(path, std::ios::binary);

```

```

129     ASSERT_TRUE(output);
130     output.write(reinterpret_cast<const char*>(bytes.data()),
131                 static_cast<std::streamsize>(bytes.size()));
132 }
133
134 const std::vector<std::uint8_t> read_back = cpu1::read_binary_file(path);
135 std::filesystem::remove(path);
136
137 EXPECT_EQ(read_back, bytes);
138 EXPECT_EQ(cpu1::checksum_bytes(read_back), cpu1::checksum_bytes(bytes));
139 }

```

Listing 13.9 bench/executable_evidence_bench.cpp

```

1 #include <cpu1/elf_fixture.hpp>
2 #include <cpu1/executable_evidence.hpp>
3
4 #include <benchmark/benchmark.h>
5
6 #include <cstdint>
7 #include <vector>
8
9 static void BMInspectSyntheticElf(benchmark::State& state)
10 {
11     const std::vector<std::uint8_t> bytes = cpu1::make_synthetic_elf();
12     for (auto _ : state) {
13         const cpu1::ElfReport report = cpu1::inspect_elf(bytes, "bench.elf");
14         benchmark::DoNotOptimize(report.sections.size());
15         benchmark::DoNotOptimize(report.symbols.size());
16         benchmark::ClobberMemory();
17     }
18 }
19
20 BENCHMARK(BMInspectSyntheticElf);
21 BENCHMARK_MAIN();

```


术语表

- 可执行文件证据** 从二进制文件、工具输出、测试和报告中得到的可复查事实，例如 checksum、machine、entry、section 和 symbol。
- ELF header** ELF 文件开头的固定结构，保存 class、endianness、machine、entry 和 section/program header 位置等信息。
- section header table** section 目录表，记录每个 section 的名字偏移、类型、flags、地址、文件偏移和大小。
- string table** 字符串表。ELF 结构里很多名字通过偏移引用它，而不是直接保存字符串。
- symbol table** 符号表，把函数或对象名字、绑定、类型、section index、地址和大小连接起来。
- diagnostic** 诊断信息。解析器遇到不支持或不安全的输入时，用 diagnostic 说明原因，而不是继续读垃圾。
- benchmark smoke** 小型 benchmark 检查。它证明 benchmark 入口能跑，但不等于完整性能结论。
- 交叉验证** 用两个独立来源检查同一事实，例如用本卷解析器和 readelf 同时验证 machine 或 section size。

索引

Application Binary Interface, 19
Executable Evidence Kit, ii, v, 2

name mangling, 21